QUALCOMM AI RESEARCH (QARE) PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 2: Classical ML & Deep Learning Core — Intuitive Study Textbook

Bias-Variance, Regularization, SVM, Trees/GBDT, GMM/EM • Backprop, Normalization, Optimizers, Losses & Metrics (ML01–ML20)

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

11 Sections Indexed

Day 4: Classical Machine Learning & Evaluation Metrics ..... p. 2

1.1 🌟 [Core — Bread & Butter] Linear, Ridge (L_2), Lasso (L_1) & ElasticNet Regression (ML01) ..... p. 2

1.2 🌟 [Core — Bread & Butter] Logistic Regression, Probability Calibration & Support Vector Machines (ML02 , ML03) ..... p. 6

1.3 🌟 [Core — Bread & Butter] Decision Trees, Random Forests & Gradient Boosting (XGBoost Math) (ML04 , ML05) ..... p. 11

1.4 🔬 [Core + Deep-Dive] Unsupervised Learning: K-Means, GMM (EM) & Dimensionality Reduction (ML06) ..... p. 17

1.5 🌟 [Core — Bread & Butter] Bias-Variance Decomposition, Cross-Validation & Imbalanced Metrics (ML07 – ML10) ..... p. 20

Day 5: Deep Learning Core Mechanics (Activations, Norms, Init, Optimizers) ..... p. 25

2.1 🌟 [Core — Bread & Butter] Activation Functions & Their Failure Modes (ML11) ..... p. 25

2.2 🌟 [Core — Bread & Butter] Normalization Layers & Inverted Dropout: Shapes, Buffers & Traps (ML12 , ML13) ..... p. 27

2.3 🌟 [Core — Bread & Butter] Weight Initialization Derivations & Symmetry Breaking (ML14 , ML15) ..... p. 31

2.4 🌟 [Core — Bread & Butter] Modern Optimizers, Decoupled Weight Decay & Training Schedules (ML16 – ML20) ..... p. 33

> ⚡ **IMPORTANT**
>
> **How to Use This Guide (Days 4–5: Nov 1 – Nov 2):** Pillar 2 is the #1 source of "**Select All That Apply**" **conceptual traps** and fast numerical exercises on HackerRank AI Residency exams (`ML01` – `ML20`). Instead of cramming dense equations into one-line bullets, every topic in this guide is broken down into four intuitive parts:
>
> 1. 💡 **Intuition (Plain English First):** What problem does this solve, and what is the physical/visual mental model? (With **inline single-concept diagrams** right where you read!)
> 2. 📐 **The Math & Every Symbol Explained:** Clean display equations where every single variable and matrix dimension ($N \times d = N$ rows/samples, d columns/features) is explicitly defined.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete small-number calculations worked out line-by-line so you can see the mechanics in action.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rule of thumb and the exact traps tested in `ML01` – `ML20`.

> 💡 **TIP**
>
> 🎯 **80/20 Bread & Butter Priority Map (Focus Here First!):**
>
> - 🌟 **[Core — Bread & Butter] (85% of Exam Marks):** Focus on **Lasso (L_1 sparsity) vs. Ridge (L_2 invertibility when $d > N$)**, **SVM margin width ($2/\|w\|_2$) & C/γ overfitting**, **Gini impurity ($2p(1-p)$) & Random Forest vs. GBDT bias-variance**, **Precision / Recall / F_1 / ROC-AUC vs. PR-AUC on imbalanced data**, **Bias-Variance MSE ($\text{Bias}^2 + \text{Var} + \sigma_\epsilon^2$)**, **Sigmoid/ReLU/GELU derivatives**, **BatchNorm vs. LayerNorm axes & Inverted Dropout $\frac{1}{1-p}$**, **He ($\sqrt{2/n_{\text{in}}}$) vs. Xavier init**, and **Adam ($t=1$ step $= -\eta\,\text{sign}(g_1)$) vs. AdamW**.
> - 🔬 **[Optional PhD Deep-Dives] (Collapsible `<details>` Blocks):** Soft-thresholding proofs, Platt vs. Isotonic calibration, XGBoost 2nd-order Taylor leaf derivations, and GMM EM matrix updates are tucked inside expandable `<details>` blocks so they never clutter your core review!

Day 4: Classical Machine Learning & Evaluation Metrics

↑ Mục Lục

1.1 🌟 [Core — Bread & Butter] Linear, Ridge (L_2), Lasso (L_1) & ElasticNet Regression (`ML01`)

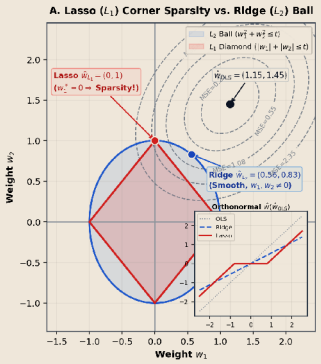
💡 Intuition (Plain English First)

Imagine you are trying to predict house prices (y) from a table of features (X) like square footage, number of bedrooms, and lot size.

- **Ordinary Least Squares (OLS)** simply draws the flat hyperplane that minimizes the sum of squared vertical distances (residuals) between the true prices and your predictions. Under ideal textbook conditions, OLS is the best unbiased linear rule you can possibly build.
- **When does OLS break?** Suppose you have **more features than data points** ($d > N$, like $1,000$ gene expression features for only 50 patients), or two features are **redundant/collinear** (e.g., Feature 1 is area in square feet and Feature 2 is area in square meters). Then there is no longer a unique best line—infinitely

many weight combinations give the exact same training predictions, and the matrix $X^T X$ becomes **singular (non-invertible)**! Even if features are only *nearly* collinear, OLS weights explode to huge positive and negative numbers (e.g., $+10,000 \times \text{sq_ft} - 107,639 \times \text{sq_m}$) that cancel out on training data but fail miserably on new test data (**massive variance / overfitting**).

- **How Regularization rescues OLS:** We add a "budget penalty" on the size of the weights w so the model isn't allowed to use giant numbers unless they dramatically reduce the error:
  - **Ridge Regression (L_2 penalty):** Penalizes the sum of **squared** weights ($\sum w_j^2$). Geometrically, the weight budget is a smooth **circle/sphere**. It shrinks all weights smoothly toward zero (especially noisy, low-variance directions) and makes $X^T X + \lambda I_d$ **100% invertible even when** $d \gg N$, but it never sets any weight to exact zero.
  - **Lasso Regression (L_1 penalty):** Penalizes the sum of **absolute** weights ($\sum |w_j|$). Geometrically, the weight budget is a **diamond with sharp corners** sitting right on the coordinate axes. When the elliptical error contours expand until they touch the diamond, they almost always hit a sharp corner where one or more coordinates are **exact zero ($w_j = 0$)**! Thus, Lasso performs automatic **feature selection (sparsity)**.
  - **ElasticNet ($L_1 + L_2$ penalty):** Pure Lasso gets confused when two features are identical twins ($x_1 \approx x_2$)—it randomly picks one and zeros out the other. By mixing the sharp corners of L_1 with the rounded edges of L_2, ElasticNet keeps the sparsity of Lasso while giving correlated twin features **nearly equal weights (the Grouping Effect)**.



Lasso L1 Diamond vs. Ridge L2 Circle Geometry & SVD Shrinkage

📐 The Math & Every Symbol Explained

1. Ordinary Least Squares (OLS):

$$\mathcal{L}_{\text{OLS}}(w) = \frac{1}{2}\|Xw - y\|_2^2$$

Setting the gradient $\nabla_w \mathcal{L}_{\text{OLS}}(w) = X^T(Xw - y) = 0$ gives the **Normal Equations**:

$$\hat{w}_{\text{OLS}} = (X^T X)^{-1} X^T y \quad (\text{requires } \text{rank}(X) = d \leq N)$$

2. Ridge Regression (L_2 Regularization):

$$\mathcal{L}_{\text{Ridge}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2 \implies \hat{w}_{\text{Ridge}} = (X^T X + \lambda I_d)^{-1} X^T y$$

3. Lasso Regression (L_1 Regularization):

$$\mathcal{L}_{\text{Lasso}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$$

Where every symbol means:

- N: Number of training samples (rows of X).
- d: Number of input features (columns of X).
- $X \in \mathbb{R}^{N \times d}$: The **design matrix** with N **rows (samples) and** d **columns (features)**.
- $y \in \mathbb{R}^{N \times 1}$: The target vector of N true outputs.
- $w \in \mathbb{R}^{d \times 1}$: The weight (coefficient) vector of d parameters we want to learn.
- $X^T X \in \mathbb{R}^{d \times d}$: The unnormalized feature covariance (Gram) matrix with d **rows and** d **columns**.
- $I_d \in \mathbb{R}^{d \times d}$: The $d \times d$ identity matrix (1s on the main diagonal, 0s everywhere else).
- $\lambda \geq 0$: Scalar regularization strength controlling how heavily we penalize large weights.
- $\|w\|_2^2 = \sum_{j=1}^{d} w_j^2 = w^T w$: Squared Euclidean ($L_2$) norm of the weights.
- $\|w\|_1 = \sum_{j=1}^{d} |w_j|$: Manhattan ($L_1$) norm of the weights.

Key Mathematical Properties to Know:

- **High-Dimensional Singularity ($d > N$):** Because the rank of a product cannot exceed the smallest dimension, $\text{rank}(X^T X) = \text{rank}(X) \leq \min(N, d)$. Whenever $d > N$, $\text{rank}(X^T X) \leq N < d$, which means the $d \times d$ matrix $X^T X$ is **rank-deficient (singular / non-invertible)** and must have at least $d - N$ zero eigenvalues!
- **Gauss-Markov Theorem:** If the true data follows $y = Xw^* + \epsilon$ with zero-mean, uncorrelated, equal-variance (homoscedastic) noise ($\mathbb{E}[\epsilon] = 0$, $\text{Cov}(\epsilon) = \sigma^2 I_N$), then $\hat{w}_{\text{OLS}}$ is **BLUE** (**B**est **L**inear **U**nbiased **E**stimator)—it has the smallest variance among all linear unbiased estimators.
- **Why Ridge is Always Invertible (Eigenvalue Shift):** Since $X^T X$ is symmetric positive semi-definite ($X^T X \succeq 0$), all of its eigenvalues satisfy $\mu_i \geq 0$. Adding λI_d shifts every eigenvalue up by λ:

  $$\mu_i(X^T X + \lambda I_d) = \mu_i + \lambda \geq \lambda > 0 \quad (\text{for any } \lambda > 0)$$

  Because no eigenvalue can ever be zero, $X^T X + \lambda I_d$ is **strictly positive definite and invertible** even when $d \gg N$ or features are 100% collinear!
- **SVD Shrinkage & Bias-Variance Tradeoff:** If we take the Singular Value Decomposition $X = U\Sigma V^T$ (with singular values σ_i), Ridge scales each principal component coordinate by:

  $$\text{Shrinkage Factor} = \frac{\sigma_i^2}{\sigma_i^2 + \lambda} < 1$$

  Directions with tiny variance ($\sigma_i^2 \ll \lambda$) get crushed toward 0. Introducing $\lambda > 0$ intentionally adds a small amount of **bias** (pulling $\mathbb{E}[\hat{w}_{\text{Ridge}}]$ toward 0) in exchange for a **much larger drop in variance**.

🔢 Step-by-Step Worked Toy Example

Suppose we have $N = 2$ houses and $d = 2$ perfectly collinear features (Feature 2 is an exact copy of Feature 1), with targets $y = [2, 4]^T$:

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} \quad (2 \times 2), \qquad y = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \quad (2 \times 1)$$

Step 1: Try Ordinary Least Squares (OLS) and watch it fail. First, compute $X^T X$ (2×2) and $X^T y$ (2×1):

$$X^T X = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 1^2 + 2^2 & 1^2 + 2^2 \\ 1^2 + 2^2 & 1^2 + 2^2 \end{bmatrix} = \begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 1(2) + 2(4) \\ 1(2) + 2(4) \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \end{bmatrix}$$

Now check the determinant of $X^T X$:

$$\det(X^T X) = (5)(5) - (5)(5) = 25 - 25 = 0$$

The eigenvalues of $X^T X$ are $\mu_1 = 10$ and $\mu_2 = 0$. Because $\det(X^T X) = 0$, $(X^T X)^{-1}$ **does not exist!** Any weights satisfying $w_1 + w_2 = 2$ (like $[2, 0]^T$, $[0, 2]^T$, or $[1002, -1000]^T$) give zero training error.

Step 2: Apply Ridge Regression with $\lambda = 1$. Add $\lambda I_2 = 1 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ to $X^T X$:

$$X^T X + \lambda I_2 = \begin{bmatrix} 5 + 1 & 5 \\ 5 & 5 + 1 \end{bmatrix} = \begin{bmatrix} 6 & 5 \\ 5 & 6 \end{bmatrix}$$

Check the new determinant and eigenvalues:

$$\det(X^T X + I_2) = (6)(6) - (5)(5) = 36 - 25 = 11 > 0 \quad (\text{eigenvalues shifted to } \mu_1 + 1 = 11,\ \mu_2 + 1 = 1)$$

Invert the 2×2 matrix using $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$:

$$(X^T X + I_2)^{-1} = \frac{1}{11} \begin{bmatrix} 6 & -5 \\ -5 & 6 \end{bmatrix}$$

Multiply by $X^T y = \begin{bmatrix} 10 \\ 10 \end{bmatrix}$ to find the unique Ridge weights:

$$\hat{w}_{\text{Ridge}} = \frac{1}{11} \begin{bmatrix} 6 & -5 \\ -5 & 6 \end{bmatrix} \begin{bmatrix} 10 \\ 10 \end{bmatrix} = \frac{1}{11} \begin{bmatrix} 60 - 50 \\ -50 + 60 \end{bmatrix} = \begin{bmatrix} 10/11 \\ 10/11 \end{bmatrix} \approx \begin{bmatrix} 0.909 \\ 0.909 \end{bmatrix}$$

Notice how Ridge cured the singularity AND automatically split the weight **equally** between the two identical features!

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Exact Soft-Thresholding Operator & ElasticNet Equations**

- **Lasso Soft-Thresholding Operator (for orthonormal features $X^TX = I_d$):** When features are orthonormal, each weight w_j decouples. If the OLS estimate for feature j is $\hat{w}_{j,\text{OLS}}$, then:
  - **Ridge** scales it multiplicatively: $\hat{w}_{j,\text{Ridge}} = \frac{\hat{w}_{j,\text{OLS}}}{1+\lambda}$ (never zero unless $\hat{w}_{j,\text{OLS}} = 0$).
  - **Lasso** shifts it toward zero by a constant amount λ and clips at zero (**Soft-Thresholding**):

    $$\hat{w}_{j,\text{Lasso}} = \text{sign}(\hat{w}_{j,\text{OLS}}) \max\left(0,\ |\hat{w}_{j,\text{OLS}}| - \lambda\right)$$

    Concrete check: If $\hat{w}_{j,\text{OLS}} = +0.40$ and $\lambda = 0.50$, Ridge outputs $\frac{0.40}{1.50} \approx +0.267$, whereas Lasso outputs $+1 \cdot \max(0,\ 0.40 - 0.50) = \mathbf{0}$!
- **ElasticNet Objective:**

  $$\mathcal{L}_{\text{ElasticNet}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\left(\alpha\|w\|_1 + \frac{1-\alpha}{2}\|w\|_2^2\right)$$

  where $\alpha \in [0, 1]$ mixes Lasso ($\alpha = 1$) and Ridge ($\alpha = 0$).

🎯 Exam Takeaway & Common Traps (`ML01`)

- **Trap 1 ($d > N$ Invertibility):** OLS $(X^TX)^{-1}$ **always fails** when $d > N$. Ridge $(X^TX + \lambda I_d)^{-1}$ **always succeeds** for any $\lambda > 0$ because its smallest eigenvalue is at least $\lambda > 0$.
- **Trap 2 (Sparsity vs. Shrinkage):** Ridge (L_2) shrinks weights asymptotically toward 0 but **never sets them to exact** 0. Lasso (L_1) produces **exact 0s** because MSE ellipses hit the sharp axis-aligned corners of the L_1 diamond.
- **Trap 3 (Bias-Variance of Regularization):** Increasing λ **increases bias** and **decreases variance**. (The Gauss-Markov theorem only guarantees OLS is best among *unbiased* estimators; a slightly *biased* Ridge estimator often achieves a lower total MSE than OLS!).

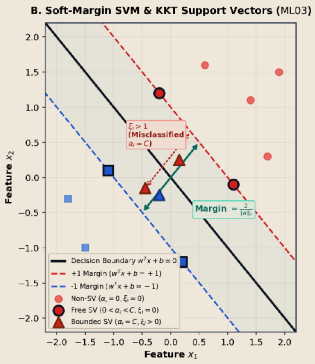
1.2 🌟 [Core — Bread & Butter] Logistic Regression, Probability Calibration & Support Vector Machines (`ML02` , `ML03`)

💡 Intuition (Plain English First)

How do we move from predicting continuous house prices to classifying emails as Spam ($y = 1$) vs. Not Spam ($y = 0$)?

- **Logistic Regression (`ML02`):** Instead of using the raw linear score $z = w^Tx + b \in (-\infty, +\infty)$ directly as a probability (which could be -5 or $+12$), we pass z through the S-shaped **Sigmoid** squishification function $\sigma(z) = \frac{1}{1+e^{-z}}$, which smoothly maps any real number into $(0, 1)$.
  - *The Linear Separability Paradox:* You might think a dataset where a line separates 100% of positive and negative points is the easiest case for Logistic Regression. **Wrong!** Because $\sigma(z)$ only *approaches* 1 as $z \to +\infty$ and 0 as $z \to -\infty$, an unregularized Logistic Regression model will keep multiplying its weights w by $10\times$, $100\times$, $1000\times$ $\to \infty$ forever trying to make the steepness infinite and push the Cross-Entropy loss all the way to 0!

- **Support Vector Machines (SVM) (`ML03`):** Unlike Logistic Regression (which cares about the probability of *every* data point, even points miles away from the boundary), an SVM acts like a civil engineer building the **widest possible safety street (margin)** between the closest positive and negative points.
  - The points sitting right on the curb of the street (or inside the street if we allow some slack) are called **Support Vectors**. If you delete every other training point far away from the street, the SVM decision boundary **does not move by a single millimeter**!
  - **What if the data isn't separable by a flat line?** Instead of explicitly computing thousands of polynomial features, the **Kernel Trick** lets us compute dot products in a high-dimensional (or even *infinite*-dimensional, for RBF) feature space using a simple similarity formula $K(x, z)$ on the original vectors.



SVM Margin Width 2/||w||_2 & KKT Support Vector Categories

📐 The Math & Every Symbol Explained

1. Logistic Regression & Log-Odds (`ML02`):

$$P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = w^T x + b$$

Taking the natural log of the ratio of positive to negative probability gives the **Log-Odds (Logit)**, which is strictly linear in x:

$$\ln\left(\frac{P(y = 1 \mid x)}{P(y = 0 \mid x)}\right) = \ln\left(\frac{\sigma(z)}{1 - \sigma(z)}\right) = w^T x + b$$

- **Binary Cross-Entropy (BCE) Loss** (for labels $y_i \in \{0, 1\}$ and predicted probability $\hat{p}_i = \sigma(w^T x_i + b)$):

  $$\mathcal{L}_{\text{BCE}}(w, b) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \ln(\hat{p}_i) + (1 - y_i) \ln(1 - \hat{p}_i) \Big]$$

  *(Equivalently, if labels are coded as $y_i \in \{-1, +1\}$, this is the **Logistic Loss**: $\frac{1}{N} \sum_{i=1}^{N} \ln\left(1 + e^{-y_i (w^T x_i + b)}\right)$).*

2. Hard-Margin & Soft-Margin SVM Primal (`ML03`): For binary labels $y_i \in \{-1, +1\}$ and decision boundary $w^T x + b = 0$:

- **Signed Geometric Distance** from point x_i to the hyperplane $w^T x + b = 0$:

  $$d_{\text{signed}}(x_i) = \frac{w^T x_i + b}{\|w\|_2}$$

- **Total Geometric Margin Width** between the positive margin curb ($w^T x + b = +1$) and negative margin curb ($w^T x + b = -1$):

$$\text{Margin Width} = \frac{2}{\|w\|_2}$$

- **Soft-Margin Primal Problem & Unconstrained Hinge Loss Form:**

$$\min_{w,b,\xi \geq 0} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{N} \xi_i \quad \text{subject to } y_i(w^T x_i + b) \geq 1 - \xi_i, \;\; \xi_i \geq 0$$

Substituting the smallest valid slack $\xi_i = \max\left(0,\; 1 - y_i(w^T x_i + b)\right)$ yields the **Hinge Loss** objective:

$$\min_{w,b} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{N} \underbrace{\max\left(0,\; 1 - y_i(w^T x_i + b)\right)}_{\text{Hinge Loss}}$$

3. SVM Dual Problem, KKT Conditions & Kernels (`ML03`):

$$\max_{0 \leq \alpha_i \leq C} \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{subject to } \sum_{i=1}^{N} \alpha_i y_i = 0$$

Once solved, the optimal weight vector in feature space is a sparse linear combination of **only the Support Vectors** ($\alpha_i > 0$):

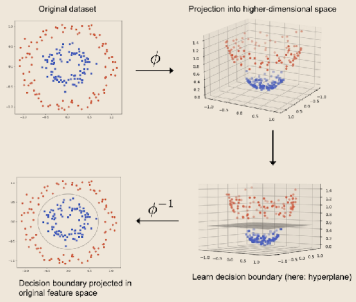
$$w^* = \sum_{i \in \text{SV}} \alpha_i y_i \phi(x_i)$$

Where every symbol means:

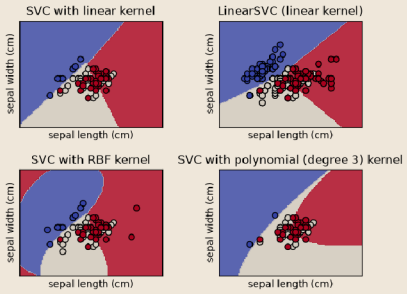
- $x_i \in \mathbb{R}^{d \times 1}$: Feature vector of sample i (d rows, 1 column).
- $y_i \in \{-1, +1\}$ (for SVM) or $\{0, 1\}$ (for BCE): Ground-truth binary class label.
- $w \in \mathbb{R}^{d \times 1}$, $b \in \mathbb{R}$: Normal vector to the hyperplane and scalar bias offset.
- $\xi_i \geq 0$: **Slack variable** for sample i, measuring how far point x_i encroaches past its margin curb $y_i(w^T x_i + b) = 1$.
- $C > 0$: **Regularization / Slack Penalty hyperparameter** (inverse of λ).
  - **Large $C \to \infty$:** Refuses to tolerate slack $\xi_i \implies$ **narrow margin**, fits training points tightly (**low bias, high variance / overfitting**).
  - **Small $C \to 0$:** Tolerates many margin violations $\implies$ **wide margin**, smooth boundary (**high bias, low variance**).
- $\alpha_i \in [0, C]$: **Dual Lagrange multiplier** for sample i. By the Karush-Kuhn-Tucker (KKT) complementary slackness conditions, every training point falls into one of three exact buckets:
  1. $\alpha_i = 0 \implies y_i(w^T x_i + b) > 1$ **($\xi_i = 0$): Non-Support Vector** strictly outside the street. Deleting this point before training changes nothing!
  2. $0 < \alpha_i < C \implies y_i(w^T x_i + b) = 1$ **($\xi_i = 0$): Free (Unbounded) Support Vector** sitting **exactly on the margin curb**. Used to solve for b^*.

3. $\alpha_i = C \implies y_i(w^T x_i + b) \leq 1$ **($\xi_i \geq 0$): Bounded Support Vector** that violates the margin (if $0 < \xi_i < 1$, it is inside the street but still on the correct side of the center line; if $\xi_i > 1$, it is **misclassified** on the wrong side of $w^T x + b = 0$).

- $K(x, z) = \langle \phi(x), \phi(z) \rangle_{\mathcal{H}}$: **Kernel function**. By **Mercer's Theorem**, a symmetric function $K(x, z)$ is a valid kernel if and only if its $N \times N$ Gram matrix $G_{ij} = K(x_i, x_j)$ is **Positive Semi-Definite ($G \succeq 0$)** for any finite set of points.
- **RBF (Gaussian) Kernel:** $K(x, z) = \exp\left(-\gamma \|x - z\|_2^2\right)$. Because the Taylor series $e^{2\gamma x^T z} = \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} (x^T z)^k$ contains all polynomial powers up to ∞, the RBF kernel maps inputs into an **infinite-dimensional** feature space! Large γ = narrow spiky Gaussian around each point = **high variance / overfitting**.



The Kernel Trick: Projecting Non-Separable 2D Data via $\phi(x)$ to a Linearly Separable 3D Space (Raschka)



SVM Decision Boundaries: Linear vs. RBF vs. Polynomial Kernels (Scikit-Learn)

🔢 Step-by-Step Worked Toy Example

Part 1: Watching Unregularized Logistic Regression Explode on Separable Data (`ML02`). Suppose we have a tiny 1D dataset with two points that are cleanly separated at $x = 0$:

- Point 1: $x_1 = -1,\ y_1 = 0$
- Point 2: $x_2 = +1,\ y_2 = 1$

Let bias $b = 0$, and let's test what happens to the Binary Cross-Entropy loss as we scale the weight $w > 0$ larger and larger:

1. **Try $w = 1$:**
   - For $x_2 = +1$ ($y_2 = 1$): $\hat{p}_2 = \sigma(1 \cdot 1) = \frac{1}{1+e^{-1}} \approx 0.731 \implies \text{Loss}_2 = -\ln(0.731) \approx 0.313$.
2. **Try $w = 5$:**
   - For $x_2 = +1$ ($y_2 = 1$): $\hat{p}_2 = \sigma(5 \cdot 1) = \frac{1}{1+e^{-5}} \approx 0.9933 \implies \text{Loss}_2 = -\ln(0.9933) \approx 0.0067$.
3. **Try $w = 20$:**

- For $x_2 = +1$ ($y_2 = 1$): $\hat{p}_2 = \sigma(20) = \frac{1}{1+e^{-20}} \approx 0.999999998 \implies \text{Loss}_2 \approx 2 \times 10^{-9}$.

4. **Why it never stops:** The gradient with respect to w is $\frac{\partial \mathcal{L}}{\partial w} = -(1 - \sigma(w)) < 0$, which is **never zero for any finite** w! Thus $\|w\|_2 \to \infty$ unless we add an L_2 penalty $\frac{\lambda}{2}w^2$ (whose positive derivative λw balances the gradient at a finite w^*).

Part 2: Computing SVM Margin Width, Hinge Loss & KKT Categories (`ML03`). Suppose a trained 2D Soft-Margin SVM with $C = 1.0$ has weight vector $w = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and bias $b = -2$.

1. **Compute the Euclidean norm $\|w\|_2$ and total geometric margin width:**

> $$\|w\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

> $$\text{Total Margin Width} = \frac{2}{\|w\|_2} = \frac{2}{5} = \mathbf{0.40}$$

2. **Classify three training points using functional margin $m_i = y_i(w^T x_i + b)$ and Hinge Loss $\xi_i = \max(0, 1 - m_i)$:**
   - **Point A:** $x_A = [2, 1]^T,\ y_A = +1$
     - Score: $w^T x_A + b = 3(2) + 4(1) - 2 = 8$.
     - Functional margin: $m_A = (+1)(8) = 8 > 1$.
     - Slack / Hinge Loss: $\xi_A = \max(0, 1 - 8) = \mathbf{0}$.
     - **KKT Status:** Strictly outside the street $\implies \alpha_{\mathbf{A}} = \mathbf{0}$ (**Non-Support Vector**).
   - **Point B:** $x_B = [1, 0]^T,\ y_B = +1$
     - Score: $w^T x_B + b = 3(1) + 4(0) - 2 = 1$.
     - Functional margin: $m_B = (+1)(1) = \mathbf{1}$.
     - Slack / Hinge Loss: $\xi_B = \max(0, 1 - 1) = \mathbf{0}$.
     - **KKT Status:** Lies **exactly on the margin curb** $\implies \mathbf{0} < \alpha_{\mathbf{B}} < \mathbf{1.0}$ (**Free Support Vector**).
   - **Point C:** $x_C = [0, 1]^T,\ y_C = -1$
     - Score: $w^T x_C + b = 3(0) + 4(1) - 2 = +2$ (predicts positive, but true label is $y_C = -1$!).
     - Functional margin: $m_C = (-1)(+2) = -2 < 1$.
     - Slack / Hinge Loss: $\xi_C = \max(0, 1 - (-2)) = \mathbf{3.0} > 1$ (misclassified!).
     - **KKT Status:** Violates the margin $\implies \alpha_{\mathbf{C}} = \mathbf{C} = \mathbf{1.0}$ (**Bounded Support Vector**).

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Implicit Bias of GD & Probability Calibration (Platt vs. Isotonic)**

- **Implicit Bias of Unregularized GD (Soudry et al.):** Even though $\|w_t\|_2 \to \infty$ when training unregularized Logistic Regression via gradient descent on linearly separable data, the *unit direction vector* $\frac{w_t}{\|w_t\|_2}$ converges to the exact **Hard-Margin SVM direction** at rate $\mathcal{O}(1/\ln t)$!
- **Probability Calibration:**
  - **Why Logistic Regression is Naturally Well-Calibrated:** Log-loss (Cross-Entropy) and Brier Score $\frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2$ are **strictly proper scoring rules**—their mathematical expectation is minimized if and only if the model outputs the true conditional probability $p_i = P(y_i = 1 \mid x_i)$.
  - **Why SVMs, Random Forests & Boosted Trees Need Calibration:**
    - **Random Forests** average predictions across many high-variance trees; to get 0.0 or 1.0, almost every single tree must agree, which pushes predicted probabilities **away from** 0 **and** 1 (under-confident; S-shaped calibration curve).
    - **Boosted Trees (GBDT) and Deep Neural Nets** push probabilities **too close to** 0 **and** 1 (over-confident).
  - **Platt Scaling (Parametric):** Fits a 1D logistic sigmoid $P(y = 1 \mid f) = \frac{1}{1+\exp(Af+B)}$ on a held-out validation set (2 scalar parameters A, B; in multiclass deep nets with $B = 0$, this is **Temperature Scaling** $\text{softmax}(z/T)$). Best when validation data is small ($N_{\text{cal}} < 1,000$).
  - **Isotonic Regression (Non-Parametric):** Fits a free-form piecewise-constant monotonically non-decreasing step function; more flexible than Platt scaling, but easily overfits unless $N_{\text{cal}} > 1,000$.

🎯 Exam Takeaway & Common Traps (`ML02` , `ML03`)

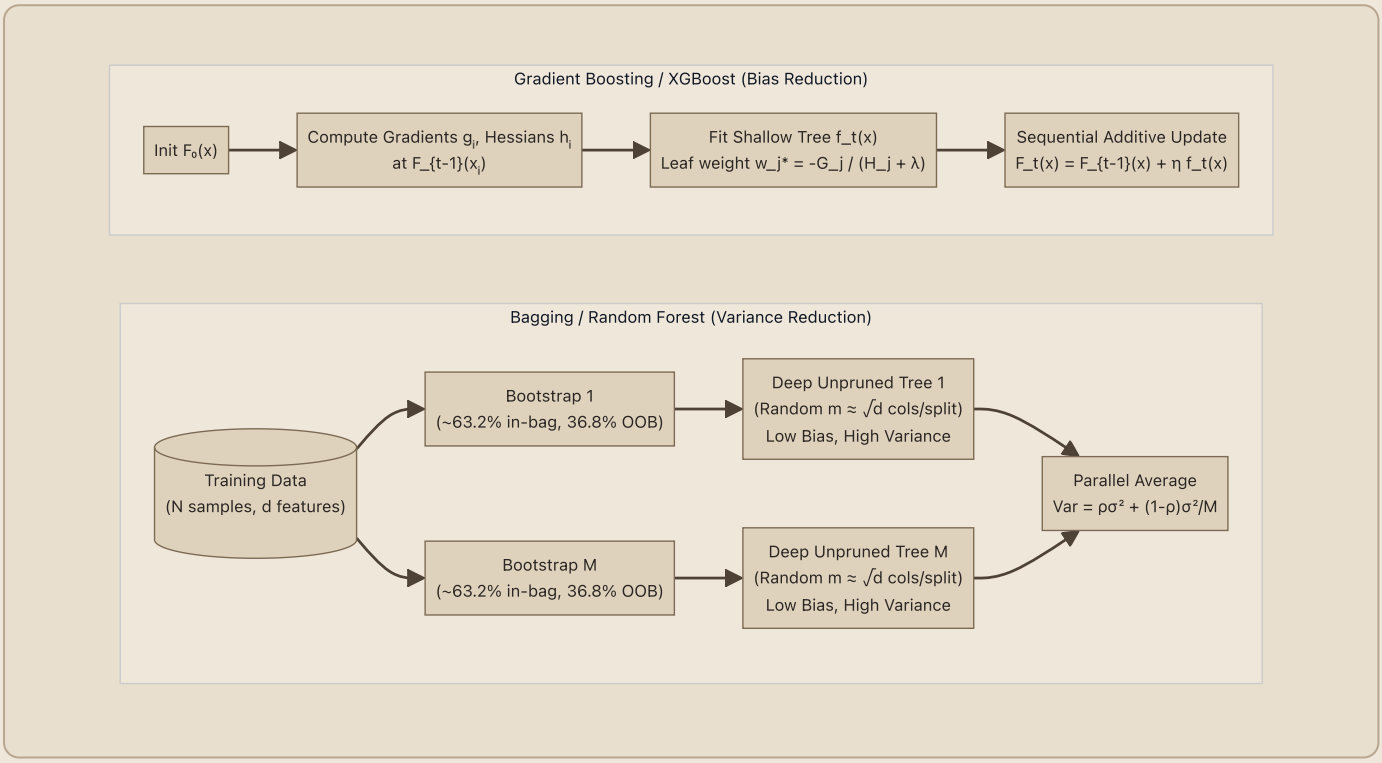
- **Trap 1 (`ML02` — Separable Data in Logistic Regression):** Without regularization ($\lambda = 0$), linearly separable data causes Logistic Regression weights to **diverge to infinity ($\|w\|_2 \to \infty$)** as BCE loss approaches 0. Adding L_2 regularization ($\lambda > 0$) guarantees a unique finite solution.
- **Trap 2 (`ML03` — Margin Width Formula):** The distance from the center line $w^T x + b = 0$ to one margin curb is $\frac{1}{\|w\|_2}$, so the **total margin width between $+1$ and -1 is** $\frac{2}{\|w\|_2}$ (don't forget the factor of 2, and don't divide by $\|w\|_2^2$!).
- **Trap 3 (`ML03` — C and γ Overfitting Direction):** Increasing $C \uparrow$ OR increasing RBF $\gamma \uparrow$ makes the SVM boundary tighter and more complex (**lower bias, higher variance / more overfitting**).

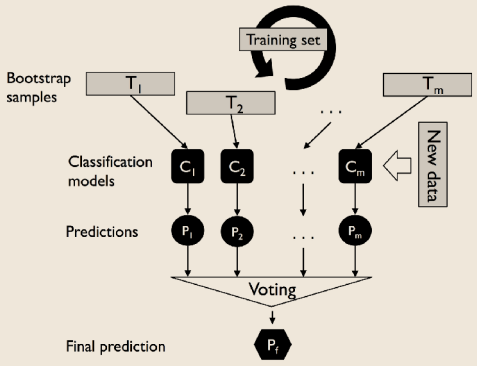
1.3 💥 [Core — Bread & Butter] Decision Trees, Random Forests & Gradient Boosting (XGBoost Math) (`ML04` , `ML05`)

💡 Intuition (Plain English First)

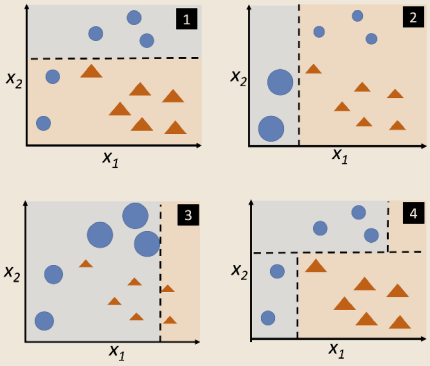
- **How a Single Decision Tree Thinks (`ML04`):** A decision tree plays "20 Questions" with your features (e.g., "*Is Age* > 30?"). At each step, it searches every feature and threshold to find the split that makes the two child groups as **pure** (homogeneous) as possible—meaning each child node contains mostly one class instead of a 50/50 coin-flip mix.
- **Why One Deep Tree Isn't Enough:** A deep, unpruned decision tree can memorize any training set (**low bias**), but if you perturb a few rows of training data, the top split can change and produce a completely different tree (**huge variance / overfitting**).

- **Two Opposite Ways to Combine Trees into an Ensemble (`ML05`):**
  1. **Bagging / Random Forest = "Wisdom of Independent Crowds" (Slashes Variance):** Take M deep, noisy, low-bias trees. Train each tree **independently in parallel** on a random bootstrap sample of rows, and force each split to only look at a random subset of $m \approx \sqrt{d}$ columns so the trees don't all pick the same dominant feature. When you average M independent noisy estimates, their random errors cancel out!
  2. **Gradient Boosting (GBDT / XGBoost) = "Relay Team of Specialists" (Slashes Bias):** Start with a weak, shallow tree (high bias, low variance, e.g., depth 3–6). Tree 2 does *not* train on the original labels y—it trains on the **mistakes (residuals / negative gradients)** left behind by Tree 1! Each subsequent tree sequentially patches the remaining errors of the whole team so far.





Bootstrap Aggregating (Bagging / Random Forest): Parallel Bootstrap Models & Majority Voting (Raschka)



Boosting: Sequential Sample Reweighting & Decision Boundary Evolution (Raschka)

📐 The Math & Every Symbol Explained

1. Node Impurity & Information Gain (`ML04`): Suppose a node contains N_{node} samples across K classes with class proportions $p_1, p_2, \ldots, p_K$ (where $\sum_{k=1}^{K} p_k = 1$):

- **Gini Impurity:** Probability that a randomly chosen sample in the node would be misclassified if we labeled it randomly according to the node's class distribution:

$$G = \sum_{k=1}^{K} p_k(1 - p_k) = 1 - \sum_{k=1}^{K} p_k^2$$

  - **3-Second Binary Shortcut ($K = 2$ with positive fraction p and negative fraction $1 - p$):**

$$G(p) = 1 - p^2 - (1 - p)^2 = 2p(1 - p)$$

    (Maximum Gini impurity for binary classification is 0.50 at a $50/50$ split $p = 0.5$; minimum is 0.00 at a pure node $p = 0$ or 1).

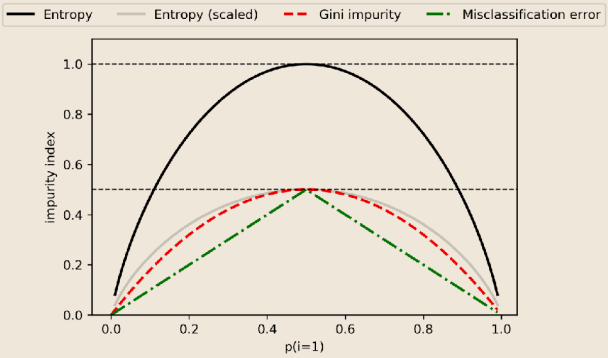
- **Shannon Entropy:**

$$H = -\sum_{k=1}^{K} p_k \log_2(p_k) \quad \text{(binary max is } \mathbf{1.0} \text{ bit at } p = 0.5)$$

- **Information Gain / Gini Gain** when splitting a Parent node (N samples) into Left (N_L samples) and Right (N_R samples):

$$\text{Gain} = I(\text{Parent}) - \left(\frac{N_L}{N} I(\text{Left}) + \frac{N_R}{N} I(\text{Right}) \right)$$

  where $I(\cdot)$ is either Gini G or Entropy H.



Decision Tree Impurity Criteria: Entropy vs. Scaled Entropy vs. Gini Impurity vs. Misclassification Error (Raschka)

- **Regression Trees:** Impurity I is the node variance (MSE). A leaf predicts the **sample mean** $\bar{y}$ of its training points when minimizing MSE (L_2 loss), or the **sample median** when minimizing MAE (L_1 loss).

2. Random Forest Math: Out-of-Bag (OOB) & Ensemble Variance:

- **Out-of-Bag (OOB) Fraction:** In bootstrap sampling, we draw N samples *with replacement* from N rows. The probability that a specific row x_i is **never picked** in all N draws is:

  $$P(\text{row } i \text{ is OOB}) = \left(1 - \frac{1}{N}\right)^N \xrightarrow{N \to \infty} e^{-1} \approx \mathbf{0.368}\ (\mathbf{36.8\%})$$

  Thus, every tree trains on $\sim 63.2\%$ **unique in-bag rows** and leaves $\sim 36.8\%$ **OOB rows** available as a built-in validation set!

- **Variance of an Average of M Correlated Trees:** If each tree has variance σ^2 and the average pairwise correlation between any two trees is $\rho \in [0, 1]$, the variance of the ensemble average $\bar{f}(x) = \frac{1}{M}\sum_{m=1}^{M} f_m(x)$ is:

  $$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$$

  - σ^2**:** Variance of a single deep tree.
  - M**:** Number of trees in the forest. As $M \to \infty$, the second term $\frac{1-\rho}{M}\sigma^2 \to 0$, so **adding more trees to a Random Forest NEVER causes overfitting**!
  - ρ**:** Pairwise correlation between trees. Even with $M = \infty$ trees, ensemble variance is floored at $\rho\sigma^2$. Restricting each split to $m \approx \sqrt{d}$ random features **decorrelates the trees ($\rho \downarrow$)**, breaking that floor!

3. Standard GBDT vs. XGBoost (`ML05`): At boosting step t, we update our additive prediction via $F_t(x_i) = F_{t-1}(x_i) + \eta f_t(x_i)$ (where η is the learning rate / shrinkage):

- **Standard GBDT (First-Order):** Fits tree f_t to the **negative gradients (pseudo-residuals)** evaluated at the previous step's predictions:

  $$r_{i,t} = -g_i = -\left[\frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i}\right]_{\hat{y}_i = F_{t-1}(x_i)}$$

- **XGBoost (Second-Order + Explicit Tree Regularization):** Approximates the loss using a **2nd-order Taylor expansion** around $F_{t-1}(x_i)$ and adds an explicit structural penalty $\Omega(f_t)$ on the tree:

$$\tilde{\mathcal{L}}^{(t)} \approx \sum_{i=1}^{N} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \underbrace{\gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2}_{\Omega(f_t)}$$

where:

- $g_i = \partial_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$: **First-order gradient** of the loss for sample i.
- $h_i = \partial^2_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$: **Second-order derivative (Hessian / curvature)** of the loss for sample i.
- T: Number of terminal leaves in tree f_t, penalized by complexity cost $\gamma \geq 0$ (acts as automatic pre-pruning!).
- w_j: Scalar prediction output of leaf j, penalized by L_2 regularization $\lambda \geq 0$.

🔢 Step-by-Step Worked Toy Example

Example 1: Computing Binary Gini Impurity & Gini Gain in 15 Seconds (`ML04`). Suppose a Parent node has $N = 10$ samples (5 Positive, 5 Negative). A candidate split divides them into:

- **Left Child ($N_L = 5$):** 4 Positive, 1 Negative ($p_L = 4/5 = 0.80$).
- **Right Child ($N_R = 5$):** 1 Positive, 4 Negative ($p_R = 1/5 = 0.20$).

Let's compute the Gini Impurity of each node and the Gini Gain line by line using $G(p) = 2p(1-p)$:

1. **Parent Gini ($p = 0.50$):**

$$G(\text{Parent}) = 2(0.50)(1 - 0.50) = 2(0.25) = \mathbf{0.50}$$

2. **Left Child Gini ($p_L = 0.80$):**

$$G(\text{Left}) = 2(0.80)(1 - 0.80) = 2(0.80)(0.20) = \mathbf{0.32}$$

3. **Right Child Gini ($p_R = 0.20$):**

$$G(\text{Right}) = 2(0.20)(1 - 0.20) = 2(0.20)(0.80) = \mathbf{0.32}$$

4. **Weighted Child Gini & Gini Gain:**

$$\text{Weighted Child Gini} = \frac{5}{10}(0.32) + \frac{5}{10}(0.32) = 0.32$$

$$\text{Gini Gain} = 0.50 - 0.32 = \mathbf{0.18}$$

Example 2: Why Feature Subsampling ($m \approx \sqrt{d}$) Matters in Random Forests. Suppose each deep tree has variance $\sigma^2 = 10.0$, and we build a forest of $M = 100$ trees:

- **Case A (Bagging without feature subsampling, high correlation $\rho = 0.60$):**

$$\text{Var}(\bar{f}) = (0.60)(10.0) + \frac{1 - 0.60}{100}(10.0) = 6.00 + 0.04 = \mathbf{6.04}$$

- **Case B (Random Forest with $m \approx \sqrt{d}$ feature subsampling, lower correlation $\rho = 0.20$):**

$$\text{Var}(\bar{f}) = (0.20)(10.0) + \frac{1-0.20}{100}(10.0) = 2.00 + 0.08 = \mathbf{2.08}$$

Notice that with $M = 100$ trees, the $\frac{1-\rho}{M}\sigma^2$ term is already tiny (0.04 vs. 0.08); almost **all** of the variance reduction ($6.04 \to 2.08$) comes from **lowering tree correlation** ρ via random feature subsampling!

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Exact XGBoost Optimal Leaf Weight & Split Gain Equations**

Let $I_j = \{i \mid x_i \in \text{leaf } j\}$ be the set of training samples falling into leaf j, and define the summed leaf gradient $G_j = \sum_{i \in I_j} g_i$ and summed leaf Hessian $H_j = \sum_{i \in I_j} h_i$. Rewriting the 2nd-order objective leaf-by-leaf gives $\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^{T} \left[G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2\right] + \gamma T$. Setting $\frac{\partial \tilde{\mathcal{L}}^{(t)}}{\partial w_j} = 0$ yields:

- **Optimal Leaf Weight (Damped Newton Step):**

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

- **Minimum Leaf Objective Value:**

$$\tilde{\mathcal{L}}_j^* = -\frac{1}{2}\frac{G_j^2}{H_j + \lambda}$$

- **Exact Split Gain (splitting parent $G_L + G_R$ into Left L and Right R):**

$$\text{Gain}_{\text{XGB}} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

(If the bracketed reduction in loss is smaller than the new-leaf complexity cost γ, then $\text{Gain}_{\text{XGB}} < 0$ and XGBoost prunes the split!)

🎯 Exam Takeaway & Common Traps (`ML04`, `ML05`)

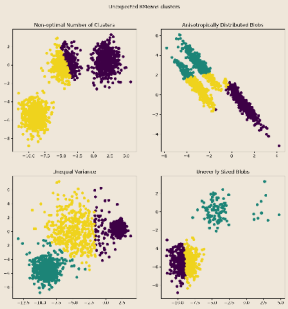
- **Trap 1 (`ML04` — Binary Gini Numbers to Memorize):** $50/50$ split $\implies G = 0.50$; $80/20$ split $\implies G = 2(0.8)(0.2) = \mathbf{0.32}$; $90/10$ split $\implies G = 2(0.9)(0.1) = \mathbf{0.18}$; pure node $\implies G = 0.00$.
- **Trap 2 (`ML05` — Bagging vs. Boosting Bias/Variance):** Random Forest starts with **deep (low-bias, high-variance)** trees and averages them in parallel to **reduce variance** (cannot overfit as $M \to \infty$). Gradient Boosting starts with **shallow (high-bias, low-variance)** trees and adds them sequentially to **reduce bias** (CAN overfit if M is too large!).
- **Trap 3 (`ML05` — GBDT vs. XGBoost):** Standard GBDT uses only **1st-order gradients (g_i)**; XGBoost uses a **2nd-order Taylor expansion (both gradients g_i and Hessians h_i)** plus explicit L_2 leaf weight (λ) and leaf count (γ) regularization.
- **Trap 4 (Feature Importance Bias):** Default tree impurity importance (**MDI**, mean decrease in Gini on training splits) is heavily biased toward **high-cardinality continuous features or ID columns**. Always use **Permutation Importance** on OOB or held-out validation data.

1.4 🔬 [Core + Deep-Dive] Unsupervised Learning: K-Means, GMM (EM) & Dimensionality Reduction (`ML06`)

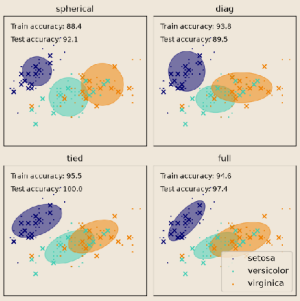
💡 Intuition (Plain English First)

What if we have no labels y at all (or we want to compress high-dimensional data before visualizing/modeling)?

- **K-Means vs. Gaussian Mixture Models (GMM):**
  - **K-Means** is a "winner-take-all" clustering algorithm. Every point belongs 100% to its single closest centroid (hard assignment), and because it measures plain Euclidean distance $\|x_i - \mu_k\|_2^2$, it implicitly assumes all clusters are **equal-radius spheres**.
  - **Why K-Means++?** If you pick initial centroids completely at random, two centroids might land in the same cluster while a faraway cluster gets zero centroids. **K-Means++** fixes this by picking the first centroid at random, and then picking each next centroid with probability proportional to its **squared distance** $D(x)^2$ from the closest already-chosen centroid!
  - **GMM (trained via Expectation-Maximization / EM)** upgrades K-Means into a **soft, probabilistic** model where each cluster k is a full Gaussian bell curve $\mathcal{N}(\mu_k, \Sigma_k)$ with its own covariance matrix Σ_k (allowing stretched, tilted **elliptical** clusters of different sizes) and each point gets a fractional responsibility $\gamma_{ik} \in (0, 1)$. In fact, K-Means is just the limiting case of GMM when all covariances are fixed to $\Sigma_k = \sigma^2 I$ and $\sigma^2 \to 0$!



When K-Means Fails: Anisotropic Blobs, Unequal Variance & Unevenly Sized Clusters (Scikit-Learn)



GMM Covariance Structures: Spherical, Diagonal, Tied & Full Elliptical Gaussians (Scikit-Learn)

- **PCA vs. LDA vs. t-SNE (`ML06`):**
  - **PCA (Principal Component Analysis): Unsupervised.** Ignores class labels completely and finds orthogonal axes of maximum total variance (spread).
  - **LDA (Linear Discriminant Analysis): Supervised.** Uses class labels to find a linear projection that simultaneously pushes **different class centers as far apart as possible** (maximizing between-class

scatter S_B) while keeping **points within the same class tightly packed** (minimizing within-class scatter S_W).

- *Why can LDA output at most $C-1$ dimensions?* Think geometry! If you have $C=2$ class centers (two points in 50D space), a single straight line ($1\text{D} = 2-1$) connects them. If you have $C=3$ class centers (three points in 50D space), they always define a flat triangle/plane ($2\text{D} = 3-1$)! Any direction perpendicular to that $(C-1)$-dimensional subspace has **zero distance between class centers**.

- **t-SNE: Non-linear manifold visualization.** In high dimensions, there is plenty of room for many neighbors at moderate distances, but when you squash them into a 2D plot, they crush together in the center (**the Crowding Problem**). t-SNE solves this by using a **heavy-tailed Student-t distribution ($\nu = 1$, Cauchy)** in 2D so moderately distant points can spread much farther apart in the 2D plot.

📐 The Math & Every Symbol Explained

1. K-Means Objective & K-Means++ Initialization:

$$\min_{\{z_{ik}\},\{\mu_k\}} \text{WCSS} = \sum_{i=1}^{N}\sum_{k=1}^{K} z_{ik}\|x_i - \mu_k\|_2^2$$

- $z_{ik} \in \{0,1\}$: Hard cluster assignment indicator (1 if point x_i is assigned to cluster k, 0 otherwise).
- $\mu_k \in \mathbb{R}^{d\times 1}$: Centroid (mean vector) of cluster k.
- **K-Means++ Sampling Probability:** Given existing centroids $\{\mu_1, \ldots, \mu_m\}$, let $D(x_i) = \min_{1\le c\le m} \|x_i - \mu_c\|_2$ be the distance from x_i to its nearest existing centroid. The next centroid is sampled with probability:

$$P(\text{select } x_i) = \frac{D(x_i)^2}{\sum_{j=1}^{N} D(x_j)^2}$$

▼ 🔬 **Optional Deep-Dive: Full GMM Expectation-Maximization (EM) Equations & Singularity Trap (Click to Expand)**

2. Gaussian Mixture Models (GMM) & EM Algorithm: Model density: $p(x_i \mid \theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)$, where $\pi_k \in [0, 1]$ are mixing weights ($\sum_{k=1}^{K} \pi_k = 1$).

- **E-Step (Expectation — compute soft responsibilities γ_{ik}):**

$$\gamma_{ik} = P(z_i = k \mid x_i, \theta^{(t)}) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

- **M-Step (Maximization — update parameters using effective cluster count $N_k = \sum_{i=1}^{N} \gamma_{ik}$):**

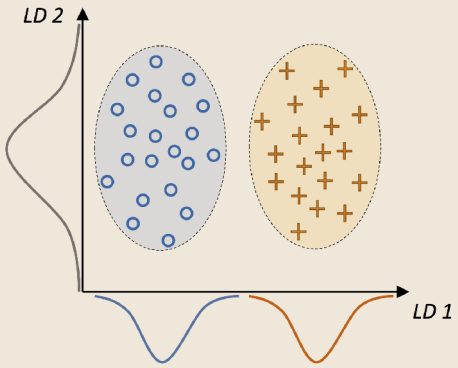
$$\pi_k^{\text{new}} = \frac{N_k}{N}, \qquad \mu_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} x_i, \qquad \Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (x_i - \mu_k^{\text{new}})(x_i - \mu_k^{\text{new}})^T$$

- **Monotonicity & The Singularity Trap:** By Jensen's Inequality (maximizing the Evidence Lower Bound, ELBO), every EM iteration is guaranteed to monotonically non-decrease the log-likelihood: $\log p(X \mid \theta^{(t+1)}) \geq \log p(X \mid \theta^{(t)})$. However, if a Gaussian component collapses onto a single data point ($\mu_k = x_i$ and $\sigma_k^2 \to 0$), the Gaussian peak $\frac{1}{\sqrt{2\pi\sigma_k^2}} \to +\infty$, causing an **infinite likelihood singularity**! We prevent this in practice by adding a small diagonal ridge $\Sigma_k \leftarrow \Sigma_k + \epsilon I_d$.

3. Linear Discriminant Analysis (LDA) Dimensionality Cap (`ML06`): Given N samples across C classes with overall mean $\mu \in \mathbb{R}^{d \times 1}$ and class means $\mu_c \in \mathbb{R}^{d \times 1}$:

- **Within-Class Scatter Matrix ($d \times d$):** $S_W = \sum_{c=1}^{C} \sum_{x_i \in c} (x_i - \mu_c)(x_i - \mu_c)^T$
- **Between-Class Scatter Matrix ($d \times d$):** $S_B = \sum_{c=1}^{C} N_c (\mu_c - \mu)(\mu_c - \mu)^T$
- **Rayleigh Quotient Objective:** $\max_W \frac{\det(W^T S_B W)}{\det(W^T S_W W)}$, solved by the top eigenvectors of $S_W^{-1} S_B$.
- **Exact Rank Bound:** Each term $(\mu_c - \mu)(\mu_c - \mu)^T$ in S_B is an outer product of a $d \times 1$ vector with itself, which has **rank at most** 1. Because the C centered vectors satisfy the linear constraint $\sum_{c=1}^{C} N_c (\mu_c - \mu) = 0$, at most $C - 1$ of them are linearly independent:

$$\text{rank}(S_B) \leq \min(d,\ C - 1) \implies \textbf{Max LDA Output Dimensions} = \min(\mathbf{d},\ \mathbf{C - 1})$$



Linear Discriminant Analysis (LDA): Maximizing Between-Class Separation Along LD1 vs. LD2 (Raschka)

🔢 Step-by-Step Worked Toy Example

Example 1: K-Means++ $D(x)^2$ Sampling Probabilities. Suppose we have 3 data points on a 1D line at $x_1 = 0,\ x_2 = 1,\ x_3 = 4$, and K-Means++ randomly picks the first centroid at $\mu_1 = x_1 = 0$. What is the exact probability of selecting x_2 vs. x_3 as the second centroid μ_2?

1. **Compute distance $D(x_i) = |x_i - \mu_1|$ to the nearest existing centroid:**
   - $D(x_1) = |0 - 0| = 0$
   - $D(x_2) = |1 - 0| = 1$
   - $D(x_3) = |4 - 0| = 4$
2. **Square each distance $D(x_i)^2$:**
   - $D(x_1)^2 = 0^2 = 0$
   - $D(x_2)^2 = 1^2 = 1$
   - $D(x_3)^2 = 4^2 = 16$
3. **Normalize by the sum $0 + 1 + 16 = 17$:**

   $$P(\mu_2 = x_2) = \frac{1}{17} \approx 5.9\%, \qquad P(\mu_2 = x_3) = \frac{16}{17} \approx \mathbf{94.1\%}$$

   Notice how squaring the distance gives the faraway point $x_3 = 4$ a $16\times$ **higher chance** of being picked than $x_2 = 1$!

Example 2: LDA vs. PCA Output Dimensions (`ML06`). Suppose a biomedical dataset has $N = 500$ patient scans, each with $d = 50$ numerical features, labeled into $C = 3$ disease classes (Healthy, Mild, Severe):

- **If you run PCA:** You can extract up to $\min(N - 1, d) = \min(499, 50) = \mathbf{50}$ principal components.
- **If you run LDA:** Because $C = 3$ class centroids in $\mathbb{R}^{50}$ span at most a 2D plane ($\text{rank}(S_B) \leq 3 - 1 = 2$), LDA can output **at most** $\min(50, 3 - 1) = \mathbf{2}$ **linear discriminants**! Asking `LinearDiscriminantAnalysis(n_components=3)` in scikit-learn will immediately raise a `ValueError`.

🎯 Exam Takeaway & Common Traps (`ML06`)

- **Trap 1 (`ML06` — LDA Max Components):** For C classes and d features, LDA produces at most $\min(d, C - 1)$ components (for binary classification $C = 2$, LDA always projects onto **1D**, regardless of whether $d = 10$ or $d = 10,000$!).
- **Trap 2 (K-Means & GMM Properties):** Both K-Means and GMM EM are **non-convex** and converge only to a **local** optimum. However, GMM MLE can diverge to **infinite log-likelihood ($+\infty$)** if a component's variance $\sigma_k^2 \to 0$ on a single data point; K-Means cannot diverge because its objective is bounded below by 0.
- **Trap 3 (t-SNE Interpretation):** t-SNE uses a heavy-tailed **Student-t ($\nu = 1$ Cauchy)** distribution in low-D to solve the Crowding Problem and minimizes asymmetric Forward KL $D_{\text{KL}}(P \parallel Q)$. **Never** interpret cluster diameters or distances between separated clusters on a t-SNE plot as true metric distances!

1.5 🌟 [Core — Bread & Butter] Bias-Variance Decomposition, Cross-Validation & Imbalanced Metrics (`ML07` – `ML10`)

💡 Intuition (Plain English First)

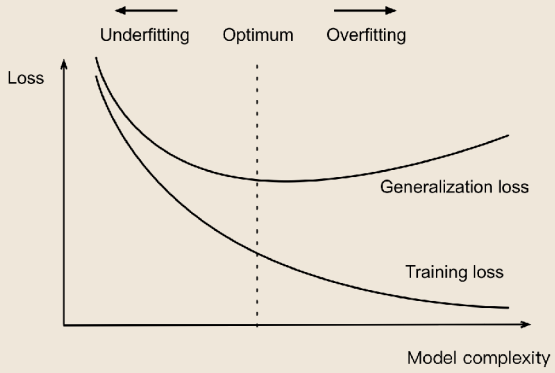
- **The Dartboard Mental Model for Bias-Variance (`ML09`):** Imagine throwing darts at a bullseye $f(x)$, where each dart is a model $\hat{f}_D(x)$ trained on a fresh random training dataset D:
  - **Bias:** How far the *average center* of your dart cloud $\mathbb{E}_D[\hat{f}(x)]$ is from the true bullseye $f(x)$. High bias = **underfitting** (your model is too simple/rigid, like fitting a straight line to a sine wave).
  - **Variance:** How widely your darts scatter around their own average center when the training set changes slightly. High variance = **overfitting** (your model memorizes random training noise).
  - **Irreducible Noise (σ_ϵ^2):** Random wind gusts inherent to the target $y = f(x) + \epsilon$ itself. Even the true function $f(x)$ suffers an expected squared error of σ_ϵ^2!
- **Why ROC-AUC Lies on Rare Events & Why PR-AUC Tells the Truth (`ML07`):** Suppose you are inspecting $10,000$ silicon wafers on a Qualcomm assembly line: 100 **are Defective (Positive)** and $9,900$ **are Normal (Negative)**. A mediocre detector catches 60 defective wafers ($TP = 60, FN = 40$) but triggers 90 false alarms ($FP = 90, TN = 9,810$):
  - **What ROC-AUC sees:** It compares $\text{TPR} = \frac{60}{100} = 60\%$ against $\text{FPR} = \frac{90}{9,900} = 0.91\%$. Because the denominator of FPR contains all $9,900$ true negatives, 90 false alarms look like a tiny drop in the ocean! The ROC curve hugs the top-left corner and reports a glowing score near 1.0.
  - **What PR-AUC sees:** It replaces FPR with $\text{Precision} = \frac{TP}{TP+FP} = \frac{60}{60+90} = \mathbf{40\%}$. Notice that $TN = 9,810$ appears **nowhere** in Precision or Recall! PR-AUC immediately exposes that 60% **of the wafers your detector flags are actually false alarms**.
- **What does ROC-AUC $= 0.85$ actually mean in plain English (`ML08`)?** It does *not* mean 85% accuracy! It means: if you pick one random positive sample x^+ and one random negative sample x^-, there is an 85% **probability that your model assigns a higher raw score to the positive sample than to the negative sample**: $P(\hat{s}(x^+) > \hat{s}(x^-)) = 0.85$. Because it only depends on **which score is bigger (pairwise rank order)**, applying *any* strictly increasing function (like $3s - 10$ or $\sigma(s)$) changes **zero** rankings and leaves ROC-AUC and PR-AUC 100% identical!

📐 The Math & Every Symbol Explained

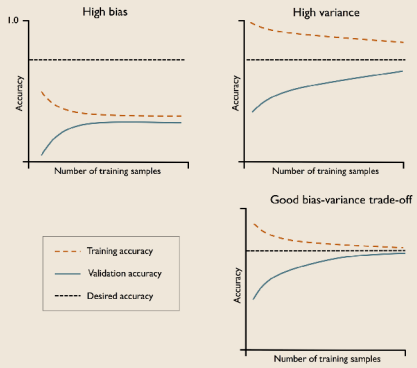
1. Exact MSE Bias-Variance Decomposition (`ML09`): Assume true target $y = f(x) + \epsilon$ where $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma_\epsilon^2$. Over random training datasets D, the expected squared test error at point x decomposes into three orthogonal terms:

$$\text{MSE}(x) = \mathbb{E}_{D,\epsilon}\left[\left(y - \hat{f}_D(x)\right)^2\right] = \underbrace{\left(\mathbb{E}_D[\hat{f}_D(x)] - f(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_D\left[\left(\hat{f}_D(x) - \mathbb{E}_D[\hat{f}_D(x)]\right)^2\right]}_{\text{Variance}} + \underbrace{\sigma_\epsilon^2}_{\text{Irreducible Noise}}$$

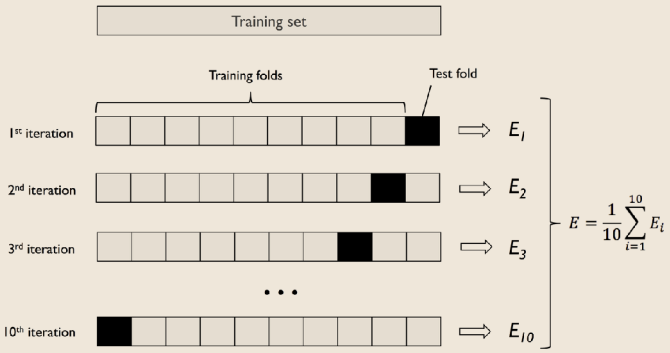
- **k-Nearest Neighbors (k-NN) Complexity Inversion:** For k-NN regression, averaging k neighbors gives $\text{Var}(\hat{f}_{k\text{-NN}}(x)) = \frac{\sigma_\epsilon^2}{k}$. Thus $k = 1$ **is the most complex model (lowest bias, highest variance σ_ϵ^2)**, and increasing $k \to N$ makes the boundary smoother (**increases bias, decreases variance**)!



Model Complexity vs. Training & Generalization Loss: Underfitting vs. Overfitting (D2L)



Learning Curves Across Training Set Size: High Bias vs. High Variance vs. Good Trade-Off (Raschka)

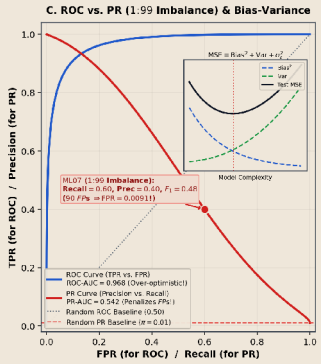


K-Fold Cross-Validation: Rotating Validation Fold & Averaged Generalization Error (Raschka)

2. Confusion Matrix Rates, F_β, MCC & Multiclass Averaging (`ML07`, `ML08`, `ML10`):

$$\text{Precision (PPV)} = \frac{TP}{TP + FP}, \qquad \text{Recall (TPR / Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{FPR (Fall-out)} = \frac{FP}{FP + TN} = 1 - \text{Specificity}, \qquad \text{Specificity (TNR)} = \frac{TN}{TN + FP}$$



ROC Curve vs. Precision-Recall Curve under Severe Class Imbalance

- **Generalized F_β Score & Matthews Correlation Coefficient (MCC):**

$$F_\beta = (1+\beta^2)\frac{\text{Precision}\cdot\text{Recall}}{(\beta^2\cdot\text{Precision})+\text{Recall}}, \qquad \text{MCC} = \frac{TP\cdot TN - FP\cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \in [-$$

  - In F_β, Recall is weighted β **times as heavily** as Precision ($\beta=1 \implies F_1$ harmonic mean; $\beta=2 \implies$ favors Recall; $\beta=0.5 \implies$ favors Precision).
  - **MCC** is the Pearson correlation coefficient between true and predicted binary labels; it uses all four cells of the confusion matrix symmetrically and equals 0 for any random or constant-class classifier.

- **Probabilistic Ranking Identity of ROC-AUC (`ML08`):**

$$\text{ROC-AUC} = P\Big(\hat{s}(x^+) > \hat{s}(x^-)\Big) + \frac{1}{2}P\Big(\hat{s}(x^+) = \hat{s}(x^-)\Big)$$

- **Multiclass Macro-F1 vs. Micro-F1 (`ML10`):** For a C-class single-label problem (where every sample has exactly 1 true class and 1 predicted class):

$$\text{Macro-}F_1 = \frac{1}{C}\sum_{c=1}^{C} F_{1,c} \quad \text{(treats all } C \text{ classes equally, even tiny rare classes!)}$$

$$\text{Micro-}F_1 = \frac{2\sum_{c=1}^{C} TP_c}{2\sum_{c=1}^{C} TP_c + \sum_{c=1}^{C} FP_c + \sum_{c=1}^{C} FN_c} = \textbf{Overall Accuracy}$$

  - *Why does* Micro-F_1 = Accuracy*?* In single-label classification, if sample i is really Class A but predicted as Class B, that single mistake counts as **one False Negative for Class A** AND **one False Positive for Class B**. Globally across all classes, $\sum_c FP_c = \sum_c FN_c = N_{\text{wrong}}$ and $\sum_c TP_c = N_{\text{correct}}$, so Micro-Precision = Micro-Recall = Micro-F_1 = $\frac{N_{\text{correct}}}{N}$ = Accuracy!

🔢 Step-by-Step Worked Toy Example

Example 1: Exact MSE Bias-Variance Calculation (`ML09`). At a test point x_0, the true noiseless function value is $f(x_0) = 10.0$, and the observation noise has standard deviation $\sigma_\epsilon = 2.0$. Across many random training sets D, our regression estimator has mean prediction $\mathbb{E}_D[\hat{f}(x_0)] = 8.5$ and prediction variance $\text{Var}_D(\hat{f}(x_0)) = 3.25$. What is the expected test MSE at x_0?

1. **Compute** Bias $= \mathbb{E}_D[\hat{f}(x_0)] - f(x_0)$ **and square it:**

$$\text{Bias} = 8.5 - 10.0 = -1.5 \implies \text{Bias}^2 = (-1.5)^2 = \mathbf{2.25}$$

2. **Identify Variance:**

$$\text{Variance} = \mathbf{3.25}$$

3. **Square the noise standard deviation $\sigma_\epsilon = 2.0$ to get Irreducible Noise variance σ_ϵ^2:**

$$\text{Irreducible Noise} = \sigma_\epsilon^2 = 2.0^2 = \mathbf{4.00}$$

4. **Sum all three components:**

$$\text{MSE}(x_0) = 2.25 + 3.25 + 4.00 = \mathbf{9.50}$$

Example 2: Pairwise Ranking & Monotonic Transform Invariance of ROC-AUC (`ML08`). Suppose we have 2 Positive samples (x_1^+, x_2^+) and 2 Negative samples (x_1^-, x_2^-) with raw model scores:

$$\hat{s}(x_1^+) = 0.8, \quad \hat{s}(x_2^+) = 0.4, \qquad \hat{s}(x_1^-) = 0.6, \quad \hat{s}(x_2^-) = 0.2$$

1. **Compute ROC-AUC by checking all $2 \times 2 = 4$ (Positive, Negative) pairs:**
   - Pair 1 (x_1^+, x_1^-): Is $0.8 > 0.6$? **Yes (1)**
   - Pair 2 (x_1^+, x_2^-): Is $0.8 > 0.2$? **Yes (1)**
   - Pair 3 (x_2^+, x_1^-): Is $0.4 > 0.6$? **No (0)**
   - Pair 4 (x_2^+, x_2^-): Is $0.4 > 0.2$? **Yes (1)**

$$\text{ROC-AUC} = \frac{1+1+0+1}{4} = \frac{3}{4} = \mathbf{0.75}$$

2. **Apply a strictly increasing transformation $g(s) = 3s - 10$ to every score:**

$$g(x_1^+) = -7.6, \quad g(x_2^+) = -8.8, \qquad g(x_1^-) = -8.2, \quad g(x_2^-) = -9.4$$

Check the 4 pairs again: $(-7.6 > -8.2)$ ✅, $(-7.6 > -9.4)$ ✅, $(-8.8 > -8.2)$ ❌, $(-8.8 > -9.4)$ ✅. The new ROC-AUC is still $\frac{3}{4} = 0.75$! *(If instead we applied a strictly decreasing function $h(s) = -3s + 10$, every inequality would flip and ROC-AUC would become $1 - 0.75 = \mathbf{0.25}$).*

🎯 Exam Takeaway & Common Traps (`ML07` – `ML10`)

- **Trap 1 (`ML07` — Extreme Class Imbalance):** When negatives vastly outnumber positives ($TN \gg TP$), **FPR and ROC-AUC are overly optimistic** because huge TN crushes $\text{FPR} = \frac{FP}{FP+TN} \approx 0$. Always choose **PR-AUC** to evaluate rare-positive detection.
- **Trap 2 (`ML08` — Monotonic Transform Invariance):** Any strictly increasing transform $g(s)$ (e.g., $3s - 10$, e^s, $\sigma(s)$) leaves **both ROC-AUC and PR-AUC** 100% **unchanged** (though it *will* change fixed-threshold F1 and calibration!).
- **Trap 3 (`ML09` — Noise Standard Deviation vs. Variance):** If an exam problem gives noise **standard deviation** $\sigma_\epsilon = 2$, remember to **square it ($\sigma_\epsilon^2 = 4$)** before adding $\text{Bias}^2 + \text{Variance} + \sigma_\epsilon^2$! Also remember: fitting `StandardScaler`, PCA, or SMOTE *before* `KFold` splitting causes **data leakage**.

- **Trap 4 (`ML10` — Macro-F1 vs. Micro-F1):** On imbalanced multiclass data, **Micro-F1 equals Overall Accuracy** (dominated by majority classes), whereas **Macro-F1 weights every class equally** $(1/C)$ and drops sharply if rare classes fail.

Day 5: Deep Learning Core Mechanics (Activations, Norms, Init, Optimizers)

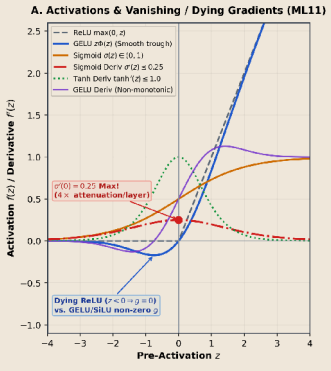
↑ Mục Lục

2.1 🌟 [Core — Bread & Butter] Activation Functions & Their Failure Modes (`ML11`)

💡 Intuition (Plain English First)

Why do we need activation functions at all? Because if you stack 50 linear layers without a non-linearity in between, matrix multiplication is associative: $W_{50}(\dots W_2(W_1 x)) = (W_{50}\cdots W_2 W_1)x = W_{\text{single}}x$. Your 50-layer deep network collapses into a single flat linear regression! However, choosing the wrong non-linear activation function introduces three famous training pathologies (`ML11`):

1. **Vanishing Gradients (Sigmoid & Tanh Saturation):** Sigmoid squashes $(-\infty, +\infty)$ into $(0, 1)$. Not only does its slope flatten to ≈ 0 at the tails ($|x| > 4$), even at its **steepest point ($x = 0$), Sigmoid's derivative is only** 0.25! During backpropagation, multiplying by ≤ 0.25 at every layer cuts the gradient by at least $4\times$ per layer.
2. **Zig-Zagging Weight Updates (Why Non-Zero-Centered Outputs Hurt):** Sigmoid outputs are strictly positive ($x_i \in (0, 1)$). When those positive numbers feed into the *next* linear layer $z = w^T x + b$, the gradient on that layer's weight vector is $\nabla_w \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial z}\right) x$. Because every entry of x is positive, **every weight gradient $\frac{\partial \mathcal{L}}{\partial w_i}$ is forced to have the exact same sign**! The optimizer can only move all weights up together $(+, +)$ or all weights down together $(-, -)$, forcing a slow diagonal zig-zag whenever the true minimum requires increasing w_1 while decreasing w_2.
3. **Dying ReLU:** $\text{ReLU}(x) = \max(0, x)$ fixes positive saturation because its slope is a clean 1.0 for all $x > 0$. But for $x < 0$, its output and slope are **exact** 0. If a large learning-rate step pushes a neuron's bias b strongly negative so that $w^T x + b < 0$ for *every* sample in the dataset, the neuron outputs 0 forever, receives 0 gradient forever, and never wakes up!



Activations & Derivatives: Sigmoid, Tanh, ReLU, GELU/Swish

📐 The Math & Every Symbol Explained

Activation	Forward Equation $f(x)$	Derivative $f'(x)$ & Range	Zero-Centered?	Key Properties & Exam Traps (`ML11`)
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x)) \in (0, \mathbf{0.25}]$	❌ No $\in (0,1)$	Max derivative is $\sigma'(0) = 0.25 \implies \geq 4\times$ gradient shrinkage per layer; all-positive output causes **zig-zagging gradients** on next layer's W!
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$	$1 - \tanh^2(x) \in (0, \mathbf{1.0}]$	✅ Yes $\in (-1,1)$	$\tanh'(0) = 1.0$ and zero-centered (fixes zig-zagging); still saturates as $\lvert x\rvert \to \infty$.
ReLU	$\max(0, x)$	$\begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases} \in \{0, 1\}$	❌ No $\in [0, \infty)$	Non-saturating for $x > 0$; suffers from **Dying ReLU** if $w^T x + b < 0$ for all inputs (0 gradient forever).
LeakyReLU / PReLU	$\max(\alpha x, x)$	$\begin{cases} 1 & x > 0 \\ \alpha & x < 0 \end{cases} \in \{\alpha, 1\}$	Nearly	Fixes Dying ReLU via small negative slope α (e.g. 0.01 in LeakyReLU, or learnable parameter in PReLU).
GELU	$x\Phi(x) \approx x\sigma(1.702x)$	Smooth non-monotonic	Nearly	Standard in BERT/GPT/ViT ($\Phi(x)$ is standard Gaussian CDF); smooth negative trough near $x \approx -0.75$ gives non-zero gradient.
Swish / SiLU	$x\sigma(x)$	$\sigma(x) + x\sigma(x)(1-\sigma(x))$	Nearly	Smooth self-gated activation used inside **SwiGLU** FFNs in modern LLMs (LLaMA, Qwen).

The Zig-Zag Gradient Equation: For a single neuron $z = w^T x + b = \sum_{i=1}^{d} w_i x_i + b$ receiving activation vector $x \in \mathbb{R}^{d \times 1}$ from the previous layer:

$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{\left(\frac{\partial \mathcal{L}}{\partial z}\right)}_{\text{scalar } \delta \in \mathbb{R}} \cdot \underbrace{x}_{\text{vector } \in \mathbb{R}^{d \times 1}} \implies \frac{\partial \mathcal{L}}{\partial w_i} = \delta \cdot x_i$$

If $x_i > 0$ for all $i = 1, \ldots, d$, then $\text{sign}\left(\frac{\partial \mathcal{L}}{\partial w_i}\right) = \text{sign}(\delta)$ is **identical for every weight** w_i!

🔢 Step-by-Step Worked Toy Example

Part 1: Watching Sigmoid Crush Gradients by $1,024\times$ in 5 Layers. Suppose we have a 5-layer network using Sigmoid activations where every layer's pre-activation is right at the origin $z^{(l)} = 0$ (the **best possible case** where Sigmoid's slope is at its maximum) and weights are $w^{(l)} = 1.0$:

1. Evaluate Sigmoid and its derivative at $z = 0$:

$$\sigma(0) = \frac{1}{1+e^0} = 0.5, \qquad \sigma'(0) = \sigma(0)\big(1 - \sigma(0)\big) = 0.5 \times 0.5 = \mathbf{0.25}$$

2. Propagate a unit gradient $\frac{\partial \mathcal{L}}{\partial a^{(5)}} = 1.0$ backward through 5 Sigmoid layers via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial z^{(1)}} = (0.25)^5 = \left(\frac{1}{4}\right)^5 = \frac{1}{1,024} \approx \mathbf{0.000977}$$

   Even with zero saturation ($z = 0$ everywhere!), the first layer's gradient is **over $1,000\times$ smaller** than the last layer's gradient!

Part 2: Concrete Numerical Proof of the Zig-Zag Weight Update (`ML11`). Suppose a neuron has current weights $w = [1.0,\ 1.0]^T$ and the true optimal weights are $w^* = [2.0,\ 0.0]^T$. To reach w^* in a straight line, we need

weight change $\Delta w^* = [+1.0,\ -1.0]^T$ (w_1 must **increase**, w_2 must **decrease**). Now suppose the input from the previous Sigmoid layer is strictly positive: $x = [0.6,\ 0.8]^T > 0$, with learning rate $\eta = 1.0$:

- **If upstream scalar gradient $\delta = \frac{\partial \mathcal{L}}{\partial z} = -1.0$:**

  $$\nabla_w \mathcal{L} = \delta \cdot x = [-0.6,\ -0.8]^T \implies \Delta w = -\eta \nabla_w \mathcal{L} = [+0.6,\ +0.8]^T \quad \text{(BOTH weights increase!)}$$

- **If upstream scalar gradient $\delta = \frac{\partial \mathcal{L}}{\partial z} = +1.0$:**

  $$\nabla_w \mathcal{L} = \delta \cdot x = [+0.6,\ +0.8]^T \implies \Delta w = -\eta \nabla_w \mathcal{L} = [-0.6,\ -0.8]^T \quad \text{(BOTH weights decrease!)}$$

  Because $x_1 > 0$ and $x_2 > 0$, **no single gradient step can ever move in the $[+, -]$ quadrant**—the optimizer must overshoot in $[+, +]$ on one step and then correct in $[-, -]$ on the next step (zig-zagging)!

🎯 Exam Takeaway & Common Traps (`ML11`)

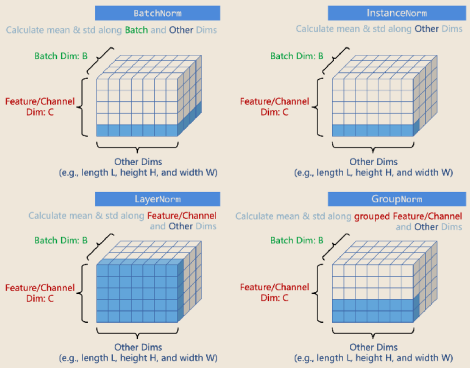
- **Trap 1 (`ML11` — Sigmoid vs. Tanh Max Slope):** Sigmoid's maximum derivative is $\sigma'(0) = 0.25$ (not 1.0!), whereas Tanh's maximum derivative is $\tanh'(0) = 1.0$ and Tanh is zero-centered.
- **Trap 2 (`ML11` — Why Non-Zero-Centered Activations Zig-Zag):** All-positive activations $x > 0$ force the gradient $\nabla_w \mathcal{L} = \frac{\partial \mathcal{L}}{\partial z} x$ of the **subsequent** linear layer to lie strictly in the all-positive or all-negative orthant.
- **Trap 3 (`ML11` — Dying ReLU vs. GELU/LeakyReLU):** Standard ReLU has an exact 0 **derivative for all $x <$ 0**, permanently killing neurons whose pre-activations become negative across the training set. LeakyReLU (slope α for $x < 0$) and GELU/SiLU (smooth negative trough) keep gradients alive.

2.2 🌟 [Core — Bread & Butter] Normalization Layers & Inverted Dropout: Shapes, Buffers & Traps (`ML12` , `ML13`)

💡 Intuition (Plain English First)

- **Why Normalize Internal Layers? (`ML12`):** As weights in Layer 1 update during training, the distribution of inputs arriving at Layer 2 shifts and scales wildly, forcing Layer 2 to constantly chase a moving target. Normalization layers pin each layer's activations back to **mean** 0 **and variance** 1, then apply a learnable scale γ and shift β so the network can undo the normalization if it wants to.
- **The Visual Classroom Analogy for `BatchNorm` vs. `LayerNorm`:** Imagine a spreadsheet where **rows are N students (samples in a batch)** and **columns are C exam questions (channels/features)**:
  - `BatchNorm` **normalizes DOWN each column (across all N students for Question c):** "*How did Student n score on Question c compared to the rest of the classroom?*" That's why BatchNorm needs a big enough classroom ($N \geq 8$) during training AND must save a `running_mean` and `running_var` notebook so that when a single student ($N = 1$) shows up alone at test time (`model.eval()`), it knows what the classroom average was!
  - `LayerNorm` **normalizes ACROSS each row (across all D questions for a single Student n):** "*How did Student n score on Question d compared to Student n's own average across all questions?*" Because LayerNorm never looks at other students in the batch, it works identically at batch size $N = 1$, needs **zero running buffers**, and has the **exact same formula in `model.train()` and `model.eval()`** !

- **Why "Inverted" Dropout? (`ML13`):** During training, Dropout randomly turns off a fraction p of neurons (keeping $1-p$ of them) to prevent co-adaptation. If you turn off 25% of neurons ($p = 0.25$) during training, the next layer only receives 75% of its usual total input energy. Instead of multiplying weights by 0.75 at test time, **Inverted Dropout divides the surviving neurons by $(1-p) = 0.75$ during training** so their expected value stays 100% unchanged and `model.eval()` **becomes a pure no-op (identity function)**!



3D Tensor Normalization Axes: BatchNorm vs. InstanceNorm vs. LayerNorm vs. GroupNorm across (B, C, H, W)

📐 The Math & Every Symbol Explained

Let's define the input tensor shapes first:

- **Computer Vision (4D Tensor):** $X \in \mathbb{R}^{N \times C \times H \times W}$, where N **= batch size (samples)**, C **= channels**, H **= spatial height**, W **= spatial width**.
- **Transformers / NLP (3D Tensor):** $X \in \mathbb{R}^{N \times L \times D}$, where N **= batch size (sequences)**, L **= sequence length (tokens)**, D **= hidden embedding dimension (features)**.

General Normalization Equation:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y = \gamma \odot \hat{x} + \beta$$

Normalization Layer	Tensor Shape	Axes Averaged for μ, σ^2	Learnable Params (γ, β)	Running Buffers (`mean`, `var`)	Key Behavior & Use Case (`ML12`)
`nn.BatchNorm2d(C)`	(N, C, H, W)	(N, H, W) per channel c ($N \cdot H \cdot W$ values)	$2C$ ($\gamma, \beta \in \mathbb{R}^C$)	$2C$ (EMA updated in `train()`, frozen in `eval()`)	Standard in CNNs; fails if $N = 1$; set preceding Conv `bias=False`!
`nn.LayerNorm(D)`	(N, L, D)	(D) per token (n, l) (D values)	$2D$ ($\gamma, \beta \in \mathbb{R}^D$)	0 (None)	Standard in Transformers; identical in `train()` and `eval()`.
`RMSNorm(D)`	(N, L, D)	(D) per token (root-mean-square only, no μ)	$1D$ ($\gamma \in \mathbb{R}^D$ only, no β)	0 (None)	Used in LLaMA/Mistral/Qwen: $y = \frac{x}{\sqrt{\frac{1}{D}\sum x_i^2 + \epsilon}} \odot \gamma$.

Normalization Layer	Tensor Shape	Axes Averaged for μ, σ^2	Learnable Params (γ, β)	Running Buffers (`mean`, `var`)	Key Behavior & Use Case (`ML12`)
`nn.InstanceNorm2d(C)`	(N, C, H, W)	(H, W) per (n, c) ($H \cdot W$ values)	$2C$ (if `affine=True`)	0 (by default)	Style Transfer; equivalent to `GroupNorm(G=C, C)`.
`nn.GroupNorm(G, C)`	(N, C, H, W)	$(C/G, H, W)$ per (n, g)	$2C$ ($\gamma, \beta \in \mathbb{R}^C$)	0 (None)	Batch-size independent CV norm; $G = 1 \iff$ LayerNorm, $G = C \iff$ InstanceNorm

Why Preceding `bias=False` Before `BatchNorm` (`ML12`): Suppose a `Linear` or `Conv2d` layer computes $z = Wx + b$. When `BatchNorm` immediately normalizes z, the mean is $\mathbb{E}[z] = \mathbb{E}[Wx + b] = \mathbb{E}[Wx] + b$. Subtracting the mean gives:

$$z - \mathbb{E}[z] = (Wx + b) - (\mathbb{E}[Wx] + b) = Wx - \mathbb{E}[Wx]$$

The bias b **cancels out algebraically to exact zero**, and BatchNorm's own learnable shift β plays the role of the bias afterward!

Inverted Dropout Equation (`ML13`): Given drop probability $p \in [0, 1)$ (so keep probability is $q = 1 - p$) and binary mask $m_i \sim \text{Bernoulli}(1 - p)$:

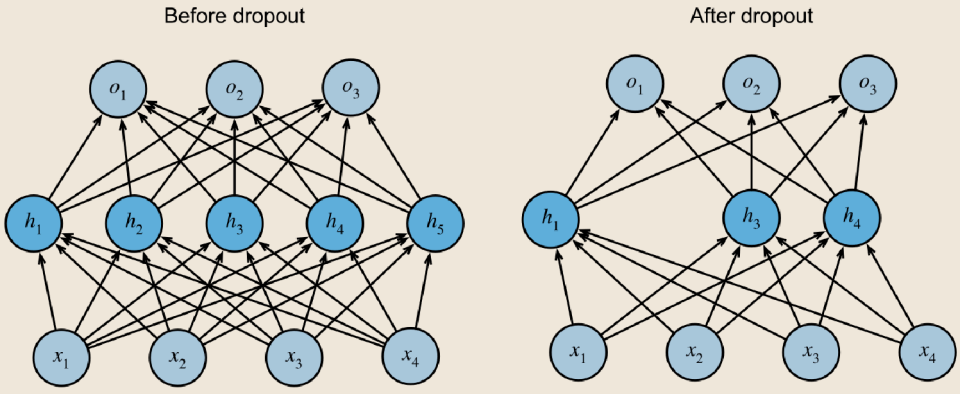
- **During `model.train()`:**

$$\tilde{a}_i = \frac{m_i \cdot a_i}{1 - p} = \begin{cases} \dfrac{a_i}{1 - p} & \text{with probability } 1 - p \\ 0 & \text{with probability } p \end{cases}$$

Checking expectation: $\mathbb{E}[\tilde{a}_i] = (1 - p)\left(\frac{a_i}{1 - p}\right) + p(0) = a_i$.

- **During `model.eval()`:**

$$\tilde{a}_i = \mathbf{1.0} \cdot \mathbf{a_i} = \mathbf{a_i} \quad \text{(exact identity — no scaling at test time!)}$$



Multilayer Perceptron Before and After Dropout Regularization (D2L)

🔢 Step-by-Step Worked Toy Example

Example 1: `BatchNorm` vs. `LayerNorm` on a 2×2 Matrix (`ML12`). Suppose we have a mini-batch of $N = 2$ samples (rows) and $D = 2$ features/channels (columns), with $\gamma = 1, \beta = 0, \epsilon = 0$:

$$X = \begin{bmatrix} 2 & 6 \\ 4 & 10 \end{bmatrix} \begin{matrix} \leftarrow \text{Sample 1} \\ \leftarrow \text{Sample 2} \end{matrix}$$

1. **`BatchNorm1d(2)` (Normalizes DOWN each column across the $N = 2$ samples):**
   - **Column 1 ($[2, 4]^T$):** Mean $\mu_1 = \frac{2+4}{2} = 3$, Variance $\sigma_1^2 = \frac{(2-3)^2+(4-3)^2}{2} = 1 \implies \sigma_1 = 1$. Normalized Column 1: $\left[\frac{2-3}{1}, \frac{4-3}{1}\right]^T = [-1, +1]^T$.
   - **Column 2 ($[6, 10]^T$):** Mean $\mu_2 = \frac{6+10}{2} = 8$, Variance $\sigma_2^2 = \frac{(6-8)^2+(10-8)^2}{2} = 4 \implies \sigma_2 = 2$. Normalized Column 2: $\left[\frac{6-8}{2}, \frac{10-8}{2}\right]^T = [-1, +1]^T$.
   - **BatchNorm Output:** $\hat{X}_{\text{BN}} = \begin{bmatrix} -1 & -1 \\ +1 & +1 \end{bmatrix}$ (Sample 1 is below the batch average on both features; Sample 2 is above on both).
2. **`LayerNorm(2)` (Normalizes ACROSS each row within each individual sample):**
   - **Row 1 ($[2, 6]$):** Mean $\mu^{(1)} = \frac{2+6}{2} = 4$, Variance $(\sigma^{(1)})^2 = \frac{(2-4)^2+(6-4)^2}{2} = 4 \implies \sigma^{(1)} = 2$. Normalized Row 1: $\left[\frac{2-4}{2}, \frac{6-4}{2}\right] = [-1, +1]$.
   - **Row 2 ($[4, 10]$):** Mean $\mu^{(2)} = \frac{4+10}{2} = 7$, Variance $(\sigma^{(2)})^2 = \frac{(4-7)^2+(10-7)^2}{2} = 9 \implies \sigma^{(2)} = 3$. Normalized Row 2: $\left[\frac{4-7}{3}, \frac{10-7}{3}\right] = [-1, +1]$.
   - **LayerNorm Output:** $\hat{X}_{\text{LN}} = \begin{bmatrix} -1 & +1 \\ -1 & +1 \end{bmatrix}$ (For both samples, Feature 2 is larger than Feature 1).

Example 2: Counting Parameters & Averages in `nn.BatchNorm2d(64)` (`ML12`). Given input tensor shape $(N = 16,\ C = 64,\ H = 32,\ W = 32)$:

- **How many scalar values are averaged to compute a single channel's μ_c?**

  $$N \times H \times W = 16 \times 32 \times 32 = \mathbf{16,384} \text{ values per channel}$$

- **How many learnable parameters (γ, β)?** $2 \times C = 2 \times 64 = \mathbf{128}$ (independent of H and W!).
- **How many non-trainable running state buffers?** $2 \times C = 2 \times 64 = \mathbf{128}$ (`running_mean` $\in \mathbb{R}^{64}$ and `running_var` $\in \mathbb{R}^{64}$).

Example 3: Inverted Dropout Calculation (`ML13`). Suppose `nn.Dropout(p=0.25)` receives activation $a = 6.0$:

- **In `model.train()`:** If the neuron is not dropped (happens with probability $1 - 0.25 = 0.75$), its output is scaled up to:

  $$\tilde{a} = \frac{6.0}{1 - 0.25} = \frac{6.0}{0.75} = \mathbf{8.0}$$

- **In `model.eval()`:** Dropout does nothing at all, outputting $\tilde{a} = \mathbf{6.0}$.

🎯 Exam Takeaway & Common Traps (`ML12`, `ML13`)

- **Trap 1 (`ML12` — `train()` vs. `eval()` Mode):** Forgetting `model.eval()` at inference time causes `BatchNorm` to normalize using current test-batch statistics (catastrophic if test batch size $N = 1$) and keeps `Dropout` randomly zeroing activations! `LayerNorm` and `GroupNorm` have **zero running buffers** and behave identically in `train()` and `eval()`.
- **Trap 2 (`ML12` — GroupNorm Equivalences):** `GroupNorm(G=1, C)` $\equiv$ `LayerNorm` over (C, H, W); `GroupNorm(G=C, C)` $\equiv$ `InstanceNorm2d(C)`.
- **Trap 3 (`ML13` — Inverted Dropout Multiplier):** Scaling by $\frac{1}{1-p}$ happens **only during training**; test-time multiplier is 1.0.

2.3 💥 [Core — Bread & Butter] Weight Initialization Derivations & Symmetry Breaking (`ML14`, `ML15`)

💡 Intuition (Plain English First)

- **The Whisper-Down-the-Hallway Analogy (`ML14`):** Imagine a 100-layer neural network as a line of 100 people passing a signal forward (and passing gradients backward).
  - If each layer's weights are slightly too large so that activation variance doubles at every layer ($\text{Var}(y) = 2\,\text{Var}(x)$), then by Layer 100 the variance is $2^{100} \approx 10^{30}$ (**Exploding Activations/Gradients** $\rightarrow$ `NaN`).
  - If each layer's weights are slightly too small so that activation variance halves at every layer ($\text{Var}(y) = 0.5\,\text{Var}(x)$), then by Layer 100 the variance is $0.5^{100} \approx 10^{-30}$ (**Vanishing Activations/Gradients** $\rightarrow$ **zero learning**).
  - **Xavier (Glorot) and Kaiming (He) Initialization** solve for the exact starting weight variance $\text{Var}(w)$ that keeps signal variance **constant** ($\text{Var}(y) = \text{Var}(x)$) from Layer 1 to Layer 100!
- **Why Does ReLU Need $2\times$ the Weight Variance of Linear/Tanh? (`ML14`):** A symmetric activation like $\tanh(z) \approx z$ near zero lets both positive and negative signals pass through. **ReLU ($\max(0, z)$) chops off the entire negative half of the bell curve and sets it to zero**, cutting the signal variance in half ($\frac{1}{2}$)! To compensate for ReLU throwing away half the variance at every layer, **Kaiming (He) Initialization doubles the weight variance** from $\frac{1}{n_{\text{in}}}$ to $\frac{2}{n_{\text{in}}}$.
- **Why Can't We Just Initialize All Weights to $W = 0$? (`ML15` — The Symmetry Trap):** In convex logistic regression (1 layer), initializing $w = 0$ works fine. In a multi-layer neural network, if you initialize a hidden layer's weight matrix to $W = 0$ (or to any constant number like 0.5), **every neuron in that hidden layer has the exact same incoming weights, computes the exact same forward activation, and receives the exact same backward gradient**! Even after $10,000$ steps of SGD, all 512 neurons in that layer remain identical clones of a single neuron. Random initialization (Xavier/Kaiming) is mandatory to **break symmetry**!

📐 The Math & Every Symbol Explained

Consider a single linear layer neuron $y = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ (with bias $b = 0$), where weights w_i and inputs x_i are mutually independent and weights have zero mean ($\mathbb{E}[w_i] = 0$):

$$\text{Var}(y) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = n_{\text{in}} \cdot \text{Var}(w) \cdot \mathbb{E}[x^2]$$

Where every symbol means:

- n_{in} (fan_in): Number of inputs feeding *into* each neuron of the layer (for `nn.Linear(in_features, out_features)`, n_{in} = in_features; for `nn.Conv2d(C_in, C_out, k, k)`, $n_{\text{in}} = C_{\text{in}} \times k \times k$).
- n_{out} (fan_out): Number of output neurons in the layer.
- $\text{Var}(w) = \sigma_w^2$: Variance of each randomly initialized weight w_i.
- $\mathbb{E}[x^2]$: Second moment (average squared value) of the input activation x coming out of the previous layer's activation function.

Case 1: Linear / Tanh / Symmetric Activation ($\mathbb{E}[x] = 0 \implies \mathbb{E}[x^2] = \text{Var}(x)$):

- Forward variance preservation ($\text{Var}(y) = \text{Var}(x)$) requires $n_{\text{in}} \text{Var}(w) = 1 \implies \text{Var}(w) = \frac{1}{n_{\text{in}}}$.
- Backward gradient variance preservation requires $n_{\text{out}} \text{Var}(w) = 1 \implies \text{Var}(w) = \frac{1}{n_{\text{out}}}$.
- Compromising via the harmonic mean gives **Xavier / Glorot Initialization**:

$$\text{Var}(w)_{\text{Xavier}} = \frac{2}{n_{\text{in}} + n_{\text{out}}} \implies \text{Normal Std: } \sigma_w = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}, \qquad \text{Uniform } [-a, a] : a = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$$

Case 2: ReLU Activation ($x = \max(0, z)$ where $z \sim \mathcal{N}(0, \text{Var}(z))$) (`ML14`):

- Because ReLU zeroes out the negative half ($z < 0$) of the zero-mean symmetric pre-activation z:

$$\mathbb{E}[x^2] = \int_{-\infty}^{0} 0^2 p(z)\, dz + \int_{0}^{\infty} z^2 p(z)\, dz = \frac{1}{2} \text{Var}(z)$$

- Substituting $\mathbb{E}[x^2] = \frac{1}{2} \text{Var}(z)$ into the forward variance equation gives $\text{Var}(y) = n_{\text{in}} \text{Var}(w) \left(\frac{1}{2} \text{Var}(z)\right)$. Setting $\text{Var}(y) = \text{Var}(z)$ requires $n_{\text{in}} \text{Var}(w) \cdot \frac{1}{2} = 1$, which gives **Kaiming / He Initialization**:

$$\text{Var}(w)_{\text{Kaiming}} = \frac{2}{n_{\text{in}}} \implies \text{Normal Std: } \sigma_w = \sqrt{\frac{2}{n_{\text{in}}}}, \qquad \text{Uniform } [-a, a] : a = \sqrt{\frac{6}{n_{\text{in}}}}$$

(Why does Uniform have a 6 instead of a 2 inside the square root? Because the variance of $\text{Uniform}(-a, a)$ *is* $\frac{(2a)^2}{12} = \frac{a^2}{3}$*, so setting* $\frac{a^2}{3} = \text{Var}(w)$ *gives* $a = \sqrt{3 \cdot \text{Var}(w)}$*!)*

🔢 Step-by-Step Worked Toy Example

Example 1: Computing Kaiming He Normal & Uniform Initialization for a ReLU Layer (`ML14`). Suppose you have a PyTorch layer `nn.Linear(in_features=512, out_features=256)` followed by a `ReLU` activation:

1. Identify $n_{\text{in}} = 512$ and $n_{\text{out}} = 256$.
2. Compute Kaiming (He) fan-in variance:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}} = \frac{2}{512} = \frac{1}{256}$$

3. Take the square root to get the **Kaiming Normal standard deviation** σ_w:

$$\sigma_w = \sqrt{\frac{1}{256}} = \frac{1}{16} = \mathbf{0.0625} \implies w_i \sim \mathcal{N}(0,\ 0.0625^2)$$

4. Multiply by $\sqrt{3}$ to get the **Kaiming Uniform bound** $[-a, a]$:

$$a = \sqrt{3 \cdot \mathrm{Var}(w)} = \sqrt{\frac{6}{512}} = \frac{\sqrt{3}}{16} \approx \mathbf{0.10825} \implies w_i \sim \mathrm{Uniform}(-0.10825,\ +0.10825)$$

Example 2: Line-by-Line Proof of the All-Zero Weight Symmetry Trap (`ML15`). Suppose a 2-neuron hidden layer $\mathrm{ReLU}(W^{(1)}x + b^{(1)})$ feeds into a 1-neuron output layer $\hat{y} = w^{(2)T}h$, and we mistakenly initialize **all weights to zero**:

$$W^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad w^{(2)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ (even if output weights are non-zero!)}$$

Let input $x = [3,\ 5]^T$ and true label $y = 10$ with loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$:

1. **Forward Pass:**
   - Neuron 1 pre-activation: $z_1 = 0(3) + 0(5) + 0 = 0 \implies h_1 = 0$.
   - Neuron 2 pre-activation: $z_2 = 0(3) + 0(5) + 0 = 0 \implies h_2 = 0$.
   - Output: $\hat{y} = 1(0) + 1(0) = 0 \implies \frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = -10$.
2. **Backward Pass (assuming LeakyReLU/Linear slope $= 1$ at 0 so gradients aren't even blocked by zero slope):**
   - Gradient arriving at Neuron 1: $\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot w_1^{(2)} = -10 \times 1 = -10$.
   - Gradient arriving at Neuron 2: $\frac{\partial \mathcal{L}}{\partial z_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot w_2^{(2)} = -10 \times 1 = -10$.
   - Weight gradient for Row 1 (Neuron 1): $\nabla_{w_1^{(1)}} \mathcal{L} = -10 \cdot [3,\ 5] = [-30,\ -50]$.
   - Weight gradient for Row 2 (Neuron 2): $\nabla_{w_2^{(1)}} \mathcal{L} = -10 \cdot [3,\ 5] = [-30,\ -50]$.
3. **Result after SGD step ($\eta = 0.01$):**

   $$W_{\text{new}}^{(1)} = \begin{bmatrix} 0.30 & 0.50 \\ 0.30 & 0.50 \end{bmatrix}$$

   Row 1 and Row 2 are still 100% identical! Neuron 1 and Neuron 2 are permanently locked together as clones forever. *(Note: Initializing **biases** $b = 0$ is completely fine and standard as long as W is initialized randomly!)*

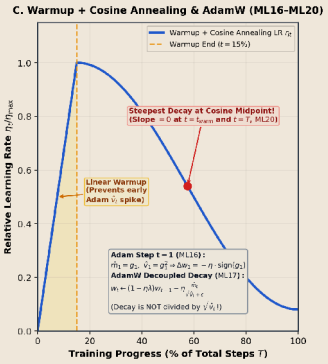
🎯 Exam Takeaway & Common Traps (`ML14`, `ML15`)

- **Trap 1 (`ML14` — ReLU Factor of 2):** Always use **Kaiming / He** ($\sigma_w = \sqrt{2/n_{\text{in}}}$) for ReLU networks, and **Xavier / Glorot** ($\sigma_w = \sqrt{2/(n_{\text{in}} + n_{\text{out}})}$) for Tanh/Sigmoid/Linear networks.
- **Trap 2 (`ML15` — Zero Weights vs. Zero Biases):** Initializing **weights** $W = 0$ (or any identical constant) fails to break symmetry and locks all hidden neurons in a layer into identical updates forever. Initializing **biases** $b = 0$ (while W is random) is completely safe and standard!

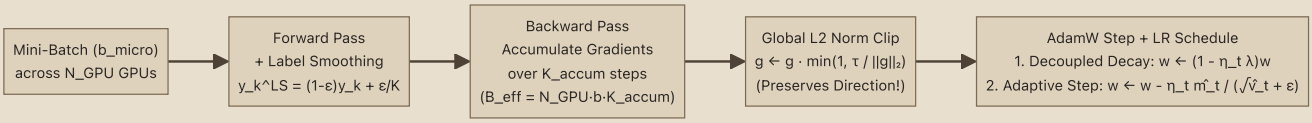
2.4 🌟 [Core — Bread & Butter] Modern Optimizers, Decoupled Weight Decay & Training Schedules (`ML16` – `ML20`)

💡 Intuition (Plain English First)

- **How Optimizers Evolve from Vanilla SGD to Adam (`ML16`):** Imagine rolling a ball down a long, narrow canyon whose side walls are super steep (high curvature) while the canyon floor slopes gently toward the minimum:
  - **Vanilla SGD** bounces wildly back and forth between the steep canyon walls while barely inching along the canyon floor.
  - **SGD with Momentum** gives the ball physical mass (v_t): the opposite wall-bounces cancel out, while the consistent downhill pushes along the canyon floor build up speed! (**Nesterov Momentum** is even smarter: it first takes the momentum step to a "lookahead" position and evaluates the gradient *there*, braking before it overshoots).
  - **AdaGrad & RMSProp** adjust the step size for each coordinate individually by dividing by the square root of past squared gradients $\sqrt{v_t}$: steep canyon walls (large g_i^2) get divided by a big number (smaller steps), while flat directions (tiny g_i^2) get divided by a small number (bigger steps). *(AdaGrad sums g_t^2 from $t = 1 \to \infty$ without forgetting, so its learning rate eventually dies to 0; **RMSProp** fixes this with a leaky moving average).*
  - **Adam (Adaptive Moment Estimation)** combines **Momentum (1st moment m_t) + RMSProp (2nd moment v_t) + Bias Correction ($\hat{m}_t, \hat{v}_t$)** (because initializing $m_0 = 0, v_0 = 0$ biases both moving averages toward zero during the first few steps!).
- **Why L_2 Regularization $\neq$ Weight Decay in Adam (`AdamW`, `ML17`):** In vanilla SGD, adding $\frac{\lambda}{2}\|w\|_2^2$ to the loss adds λw to the gradient, which subtracts $\eta\lambda w$ from the weights—identical to decaying weights by $(1 - \eta\lambda)w$. **In Adam, they are NOT equivalent!** If you add $\frac{\lambda}{2}\|w\|_2^2$ to the loss, the penalty λw gets mixed into the gradient g_t and then **divided by the adaptive preconditioner** $\sqrt{\hat{v}_t}$! That means weights with large historical gradients get almost *zero* weight decay, while weights with tiny gradients get heavily decayed. **AdamW** fixes this by **decoupling** weight decay from the gradient and shrinking w directly by $(1 - \eta_t\lambda)w$!
- **Label Smoothing (`ML18`), Effective Batch Size (`ML19`) & LR Schedules/Clipping (`ML20`):**
  - **Label Smoothing:** Instead of forcing softmax to chase hard $[0, 1, 0]$ targets (which drives logits $z_k \to +\infty$ and makes the model over-confident), we shave a tiny ϵ (e.g., 0.1) off the 1.0 and spread it evenly ($\frac{\epsilon}{K}$) across all K classes.
  - **Effective Batch Size:** If each GPU can only fit $b_{\text{micro}} = 8$ samples in VRAM, we can simulate a massive batch size by running across N_{GPU} GPUs in parallel AND accumulating gradients over K_{accum} forward/backward passes before calling `optimizer.step()`.
  - **Global L_2 Norm Clipping vs. Value Clamping:** If a bad mini-batch produces a giant gradient vector g, clamping each coordinate independently to $[-\tau, +\tau]$ **changes the angle/direction** of the vector! Rescaling the whole vector by $\frac{\tau}{\|g\|_2}$ shrinks its length to τ while **preserving its exact direction**.



Linear Warmup + Cosine Annealing LR Schedule & AdamW Decoupled Decay



📐 The Math & Every Symbol Explained

1. Momentum vs. Nesterov Accelerated Gradient (NAG):

- **Standard Momentum:** $v_{t+1} = \mu v_t - \eta \nabla \mathcal{L}(w_t), \qquad w_{t+1} = w_t + v_{t+1}$ (gradient evaluated at **current** w_t).
- **Nesterov (NAG):** $v_{t+1} = \mu v_t - \eta \nabla \mathcal{L}(w_t + \mu v_t), \qquad w_{t+1} = w_t + v_{t+1}$ (gradient evaluated at **lookahead** $w_t + \mu v_t$).

2. Adam & AdamW Update Equations (`ML16`, `ML17`): Given gradient $g_t = \nabla_w \mathcal{L}_t(w_{t-1})$ at step $t \geq 1$ (initialized with $m_0 = 0,\ v_0 = 0$):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \qquad \text{(1st moment EMA of gradient, default } \beta_1 = 0.9\text{)}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \qquad \text{(2nd moment EMA of squared gradient, default } \beta_2 = 0.999\text{)}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \qquad \text{(Bias-corrected 1st and 2nd moments at step } t\text{)}$$

$$w_t = \underbrace{(1 - \eta_t \lambda) w_{t-1}}_{\text{Decoupled Weight Decay (AdamW)}} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

3. Label Smoothing Cross-Entropy (`ML18`): For K classes, smoothing parameter $\epsilon \in [0, 1]$, and hard one-hot label $y_k \in \{0, 1\}$:

$$y_k^{\text{LS}} = (1 - \epsilon) y_k + \frac{\epsilon}{K} = \begin{cases} (1 - \epsilon) + \dfrac{\epsilon}{K} & \text{if } k = \text{true class } c \\ \dfrac{\epsilon}{K} & \text{if } k \neq c \end{cases}$$

4. Effective Batch Size (`ML19`), Cosine Annealing & Gradient Clipping (`ML20`):

- **Effective Batch Size:**

$$B_{\text{eff}} = N_{\text{GPU}} \times b_{\text{micro}} \times K_{\text{accum}}$$

(When scaling B_{eff} by $k\times$, gradient variance drops by $1/k$; scale LR linearly $\eta \leftarrow k\eta$ for SGD, or by $\sqrt{k}$ for AdamW).

- **Cosine Annealing LR Schedule** (over $t \in [0, T]$ steps):

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T}\pi\right)\right)$$

  - Derivative $\frac{d\eta_t}{dt} \propto -\sin\left(\frac{t}{T}\pi\right)$: decay rate is 0 **(flat)** at $t = 0$ and $t = T$, and **steepest in the middle at** $t = T/2$ (where $\eta_{T/2} = \frac{\eta_{\max}+\eta_{\min}}{2}$)!

- **Global L_2 Norm Clipping (`torch.nn.utils.clip_grad_norm_`):**

$$g \leftarrow g \cdot \min\left(1,\ \frac{\tau}{\|g\|_2}\right) = \begin{cases} g & \text{if } \|g\|_2 \le \tau \\ \frac{\tau}{\|g\|_2} g & \text{if } \|g\|_2 > \tau \end{cases}$$

🔢 Step-by-Step Worked Toy Example

Example 1: The Adam Step $t = 1$ Exam Shortcut (`ML16`). Suppose a scalar weight starts at $w_0 = 2.0$ with $\eta = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 0$, and the first gradient at $t = 1$ is $g_1 = +5.0$ (with $m_0 = 0, v_0 = 0$). What is w_1?

1. **Compute biased moments m_1 and v_1:**

$$m_1 = 0.9(0) + (1 - 0.9)(5.0) = 0.1 \times 5.0 = \mathbf{0.5}$$

$$v_1 = 0.999(0) + (1 - 0.999)(5.0^2) = 0.001 \times 25.0 = \mathbf{0.025}$$

(Notice how tiny $m_1 = 0.5$ and $v_1 = 0.025$ are compared to the true $g_1 = 5.0$ and $g_1^2 = 25.0$—that's why bias correction is needed!)

2. **Apply Bias Correction at $t = 1$ ($1 - \beta_1^1 = 0.1$ and $1 - \beta_2^1 = 0.001$):**

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^1} = \frac{0.5}{0.1} = \mathbf{5.0} = g_1$$

$$\hat{v}_1 = \frac{v_1}{1 - \beta_2^1} = \frac{0.025}{0.001} = \mathbf{25.0} = g_1^2$$

3. **Compute the parameter update Δw_1:**

$$\Delta w_1 = -\eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = -0.01 \times \frac{5.0}{\sqrt{25.0}} = -0.01 \times \frac{5.0}{5.0} = \mathbf{-0.01} = -\eta \cdot \text{sign}(g_1)$$

$$w_1 = w_0 + \Delta w_1 = 2.0 - 0.01 = \mathbf{1.99}$$

> **3-Second Exam Rule (`ML16`):** At step $t = 1$, $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$ for **any** $\beta_1, \beta_2 \in (0, 1)$, so the very first Adam step is **always** $\Delta w_1 = -\eta \cdot \text{sign}(g_1)$, completely independent of β_1, β_2, or the magnitude of g_1!

Example 2: Label Smoothing Target Vector (`ML18`). For $K = 5$ classes, smoothing $\epsilon = 0.10$, and true class index $c = 2$ (hard one-hot $y = [0, 1, 0, 0, 0]$):

- **True class ($k = 2,\ y_2 = 1$):**

$$y_2^{\text{LS}} = (1 - 0.10)(1) + \frac{0.10}{5} = 0.90 + 0.02 = \mathbf{0.92}$$

- **Each of the 4 wrong classes ($k \neq 2,\ y_k = 0$):**

$$y_k^{\text{LS}} = (1 - 0.10)(0) + \frac{0.10}{5} = \mathbf{0.02}$$

- **Smoothed target vector:** $y^{\text{LS}} = [0.02,\ \mathbf{0.92},\ 0.02,\ 0.02,\ 0.02]$ (check sum: $0.92 + 4(0.02) = 1.00$).

Example 3: Effective Batch Size (`ML19`) & Global Norm Clipping vs. Value Clamping (`ML20`).

- **Effective Batch Size:** Training on $N_{\text{GPU}} = 4$ GPUs with micro-batch size $b_{\text{micro}} = 8$ per GPU and `gradient_accumulation_steps` $K_{\text{accum}} = 16$:

$$B_{\text{eff}} = 4 \times 8 \times 16 = \mathbf{512}$$

- **Global L_2 Norm Clipping vs. Element-Wise Clamping:** Suppose gradient vector $g = [6.0,\ 8.0]^T$ (note the direction ratio $\frac{g_2}{g_1} = \frac{8}{6} = \frac{4}{3}$) and threshold $\tau = 5.0$:
  1. Compute L_2 norm: $\|g\|_2 = \sqrt{6.0^2 + 8.0^2} = \sqrt{36 + 64} = \sqrt{100} = 10.0 > 5.0$.
  2. **Global Norm Clip (`clip_grad_norm_`):** Scale entire vector by $\frac{\tau}{\|g\|_2} = \frac{5.0}{10.0} = 0.5$:

$$g_{\text{norm-clipped}} = 0.5 \times [6.0,\ 8.0]^T = [\mathbf{3.0},\ \mathbf{4.0}]^{\mathbf{T}} \quad \left(\text{norm} = 5.0,\ \text{direction ratio } \frac{4}{3} \textbf{ preserved!}\right)$$

  3. **Element-Wise Value Clamp (`clip_grad_value_`):** Clamp each coordinate independently to $[-5.0, +5.0]$:

$$g_{\text{val-clamped}} = [\min(6.0, 5.0),\ \min(8.0, 5.0)]^T = [\mathbf{5.0},\ \mathbf{5.0}]^{\mathbf{T}} \quad \left(\text{direction ratio changed to } \frac{5}{5} = 1 \text{ — } \textbf{disto}$$

🎯 Exam Takeaway & Common Traps (`ML16` – `ML20`)

- **Trap 1 (`ML16` — Adam Step $t = 1$):** Because bias correction divides by $(1 - \beta_1^1)$ and $(1 - \beta_2^1)$, Adam's first update is always $-\eta \cdot \text{sign}(g_1)$. Without bias correction, early steps are biased **toward** 0 (not ∞!).
- **Trap 2 (`ML17` — Adam vs. AdamW):** In Adam, L_2 loss regularization $\frac{\lambda}{2}\|w\|_2^2$ is **not** equivalent to weight decay because the penalty λw gets divided by $\sqrt{\hat{v}_t}$ (giving large-gradient weights weaker decay). AdamW decouples weight decay directly onto w.
- **Trap 3 (`ML20` — Warmup, Cosine & Clipping):**
  - **Linear Warmup** prevents early divergence in Adam/Transformers when $\hat{v}_t$ is estimated from only a handful of mini-batches and has high variance.
  - **Cosine Annealing** has slope 0 at $t = 0$ and $t = T$, and drops **steepest at the midpoint** $t = T/2$.
  - **Global L_2 Norm Clipping** rescales $g \leftarrow \frac{\tau}{\|g\|_2} g$ and **preserves gradient direction**, whereas element-wise value clamping distorts direction.